

考试安排与复习

日期：2025年12月



北京邮电大学

内容

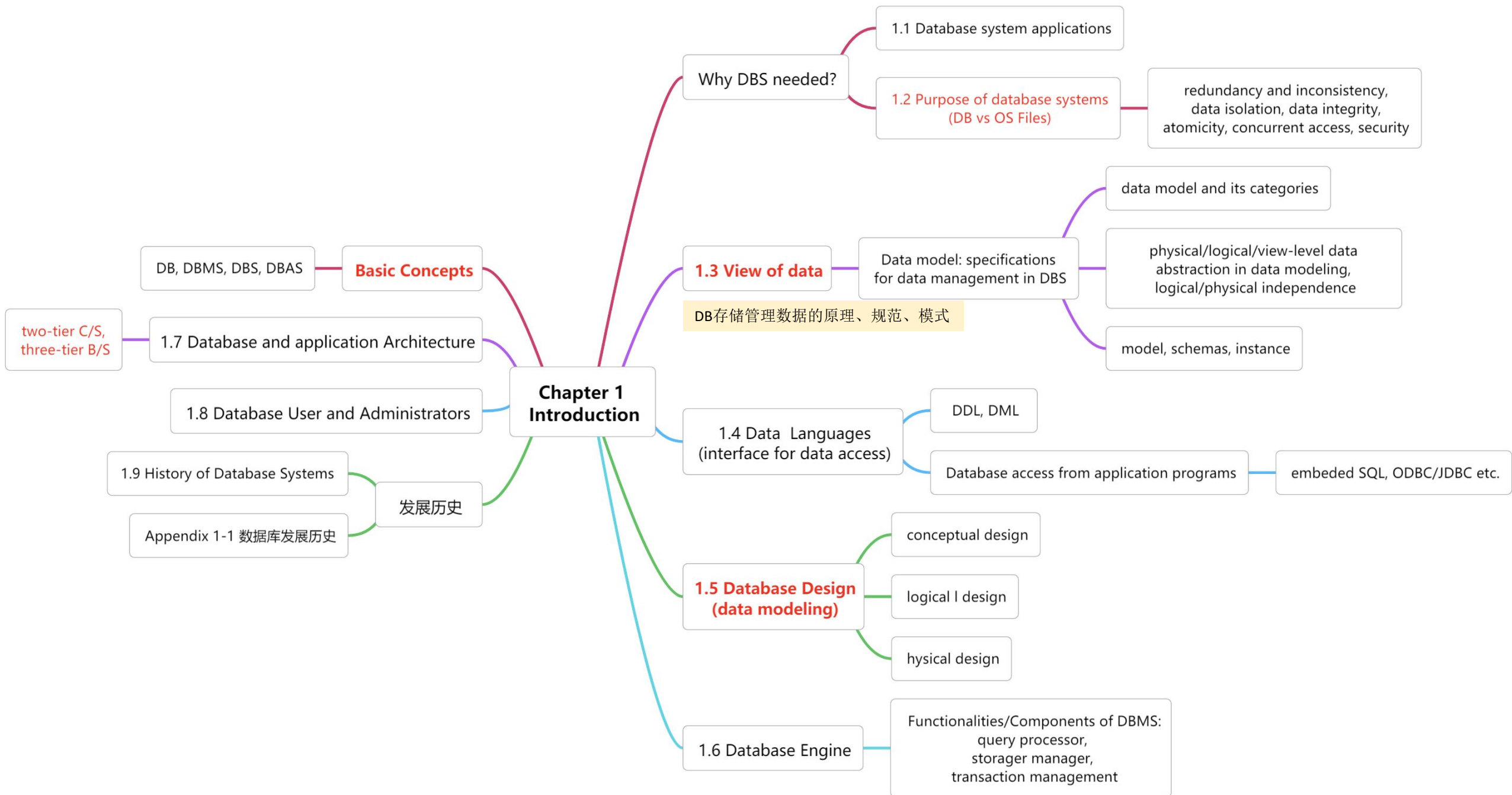
- 考试及答疑安排
- 考试内容
 - ▣ 范围
 - ▣ 内容/形式
- 各章要点总结

考试及答疑安排

- 考试时间、地点
 - 2025年12月29号，周一，8:00~10:00
 - 教三-328
- 答疑安排
 - 课程微信群
 - 2025年12月24号，周三，14:30~16:00，教三-437
- 闭卷考试，遵守线上考试纪律

考试内容

- 考试范围
 - ▣ 概念：第1章Introduction~第19章
 - ▣ 其它：第7章Relational Database Design，第13章Data Storage Structures~第19章Recovery
- 试题类型
 - ▣ 选择题
 - ▣ 简答
 - ▣ 计算
 - ▣ 设计分析
- 形式
 - ▣ 课堂例题，课后作业，期中测验
- 英文试题，中文答题



Chapter 1 Introduction

■ DBS概念和组成

▣ DB, DBMS, DBS, DBAS

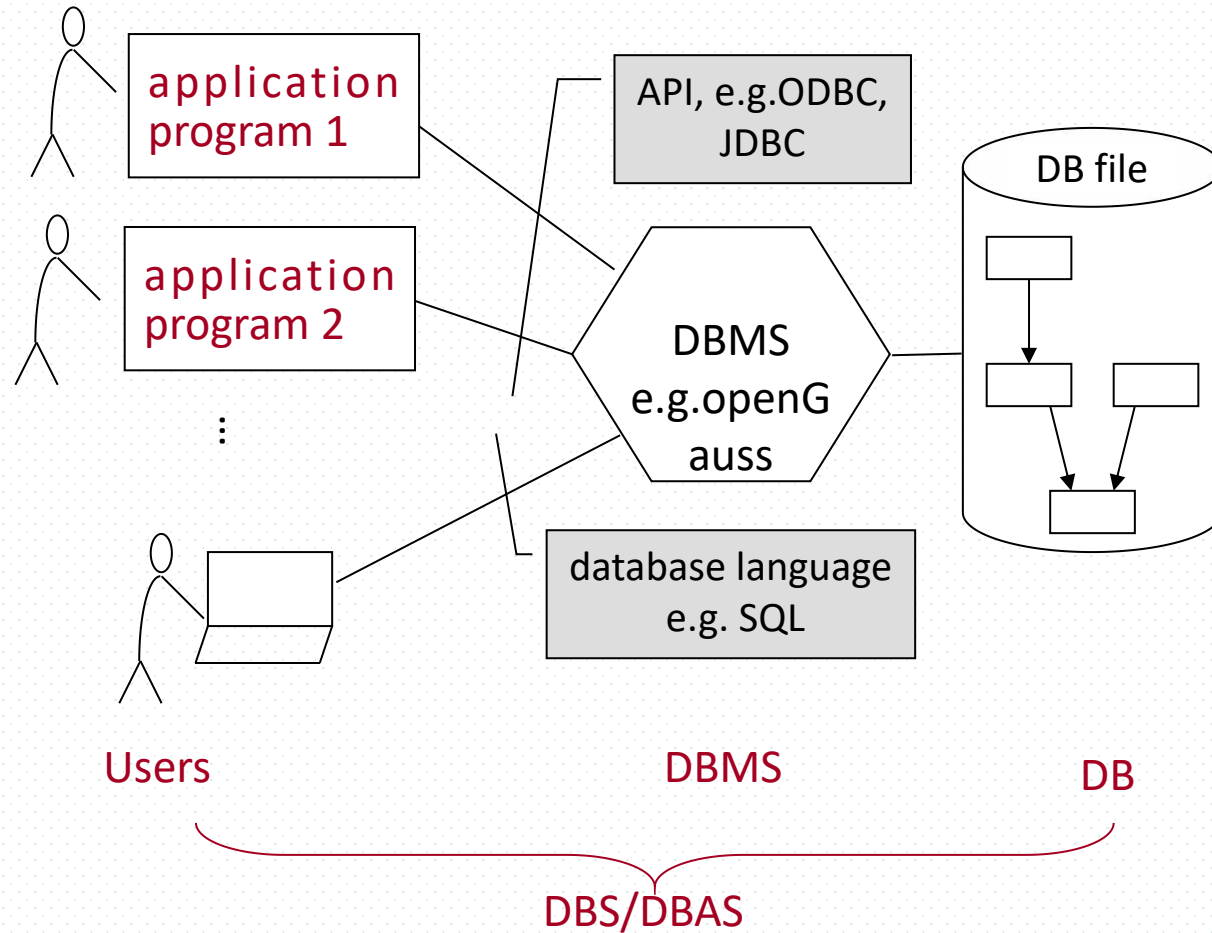


Fig. 1.0.1 DBS and DBAS

数据模型与数据抽象

- Data model
 - ▣ a collection of conceptual tools for describing data, data relationship, data semantics, consistency
- 数据模型

DBS中对数据组织与管理/使用方式的抽象描述，包括

 - ▣ 数据组织的语法定义，如数据项、数据项间的联系
 - ▣ 数据组织的语义定义，如完整性约束
 - ▣ 数据操作 (*note: only in some data models, e.g. relational data model*)

Levels of Abstraction

■ Physical level

- ▣ describes how a record (e.g., instructor) is stored, e.g. heap file, index

■ Logical level

- ▣ Implementation/DB-oriented
- ▣ describes data stored in database, and the relationships among the data.

type *instructor* = **record**

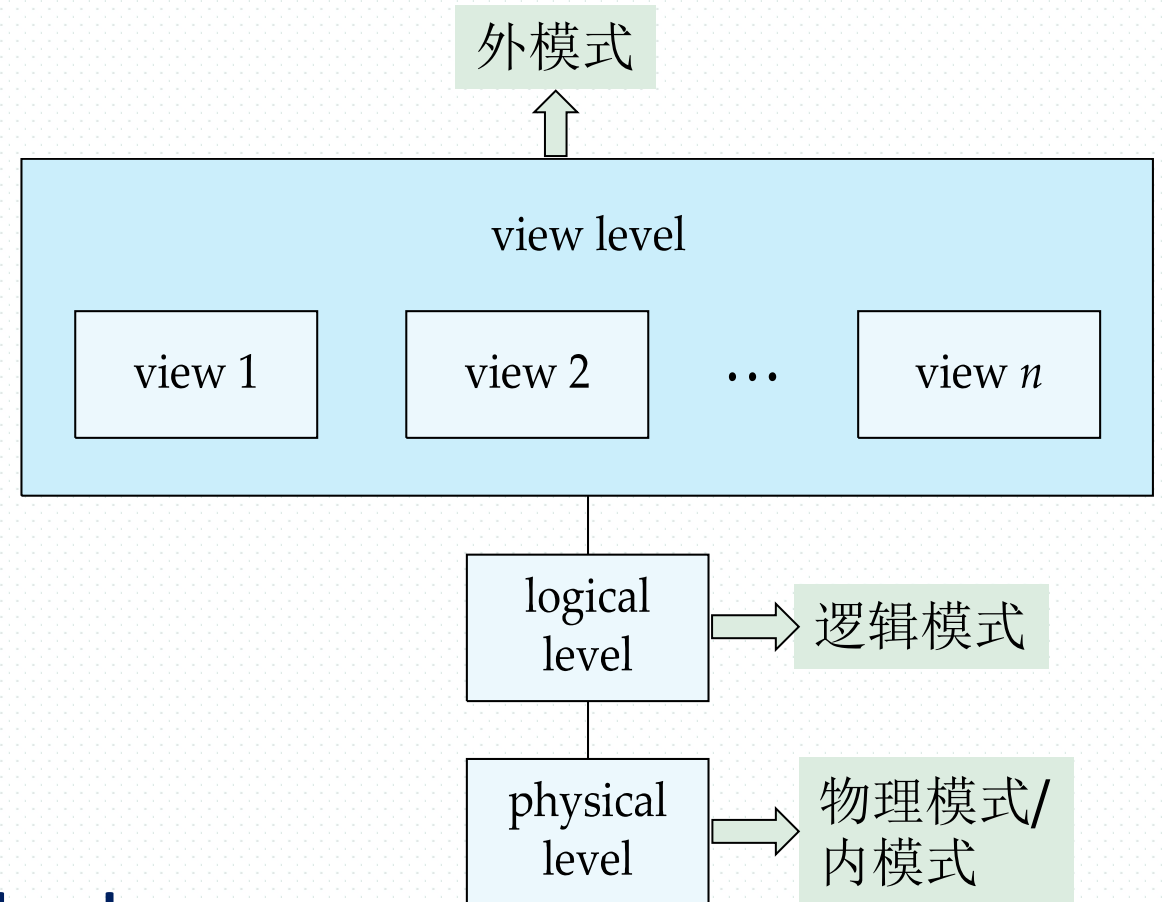
ID : string;

name : string;

dept_name : string;

salary : integer;

end;



■ View level

- ▣ application programs hide details of data types
- ▣ views can also hide information (such as an employee's salary) for security purposes.

Model, Schemas, Instances

- Data modeling, in line with data models, produces data **schemas**(模式), including the logical and physical schema, which describe the organization of data in database
- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of **customers** and **accounts** in a **bank** and the **relationship** between them
 - analogous to type information of a variable in a program
- **Physical schema** – the overall physical structure of the database
 - e.g how data is stored in database file, and how to access the database file
- **DB Instance** – the actual content of the database at a particular point in time
 - analogous to the value of a variable
- **Model** vs **schemas** vs **instance**
 - similar to types, variables and values of variables in programming languages
 - **int** **x** = **0**

Model, Schemas, Instances

■ E.g.

- ▣ relational data model 
 - $R = \{ \langle a_1, a_2, \dots, a_n \rangle \}$
- ▣ relational data schema
 - instructor = $\langle \text{ID}, \text{name}, \text{dept_name}, \text{salary} \rangle$
- ▣ instance of schema
 - $\langle 2222, \text{Einstein}, \text{Physics}, 95000 \rangle$

Data Independence

- Independence between data uses/application programs and data in DBS
- Logical data independence
 - when DBS's logical schemas at logical level (DBMS-oriented) change, application-oriented external schema or application programs do not change
 - e.g.
student=<*id*, *name*, *dept_name*, *tot_cred*>
is extended into
student
=<*id*, *name*, ***age***, *dept_name*, *tot_cred*>,
or changed into
student
=<***sid***, ***sname***, *age*, *dept_name*, *tot_cred*>
- Physical data independence
 - application programs do not depend on DBS's physical schemas
 - i.e. when physical schemas such as *index*, *storage location*, *DB file* and *block-size* change, application programs remain unchanged
 - the ability to modify the physical schema without changing the external/logical schemas, on which application programs depend
- Database Migration (数据库迁移)
 - change/replace DBMS platform, e.g. from Oracle to openGauss

Data Independence

■ 逻辑模式→内模式间的映射与physical data independence

- 物理存储发生变化, **调整**逻辑模式→内模式间的映射, 保证逻辑模式不变, 应用程序尽可能不变
- 由DBMS提供

■ 外模式→逻辑模式间的映射与logical data independence

- 数据逻辑结构发生变化, 如关系表发生变化, **调整**外模式→逻辑模式间的映射, 保证外模式不变, 从而应用程序尽可能不变
 - 应用程序基于外模式访问数据库

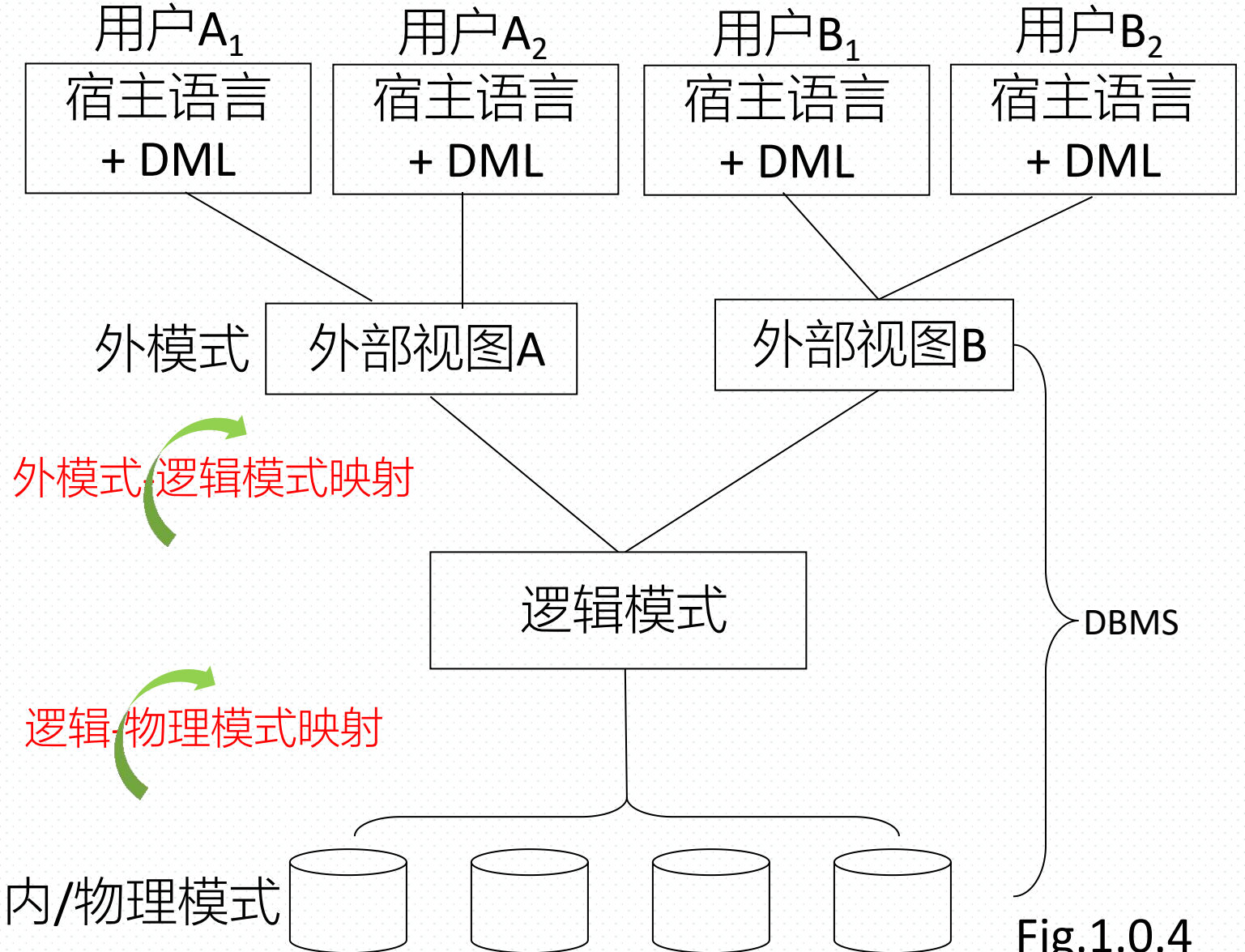


Fig.1.0.4

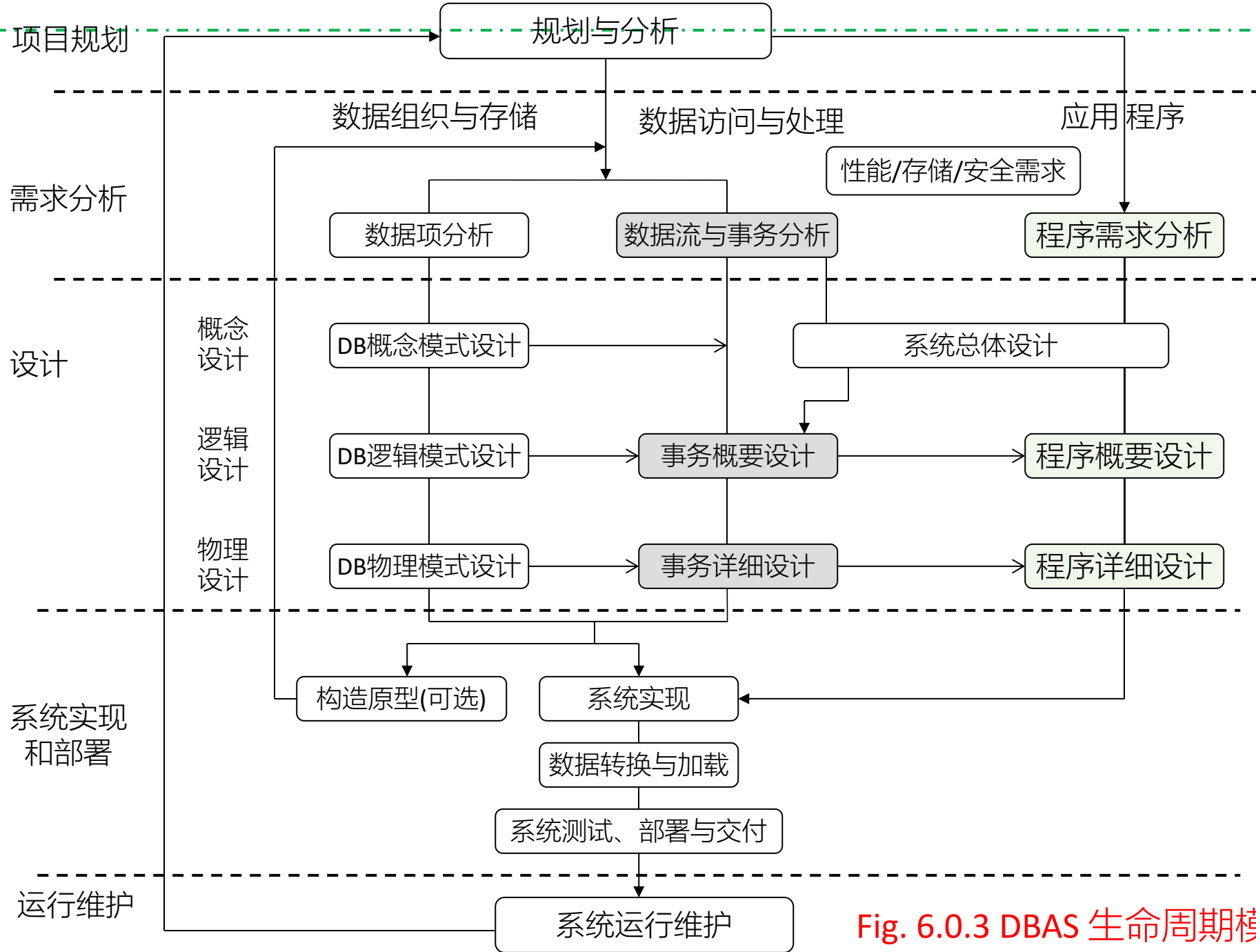
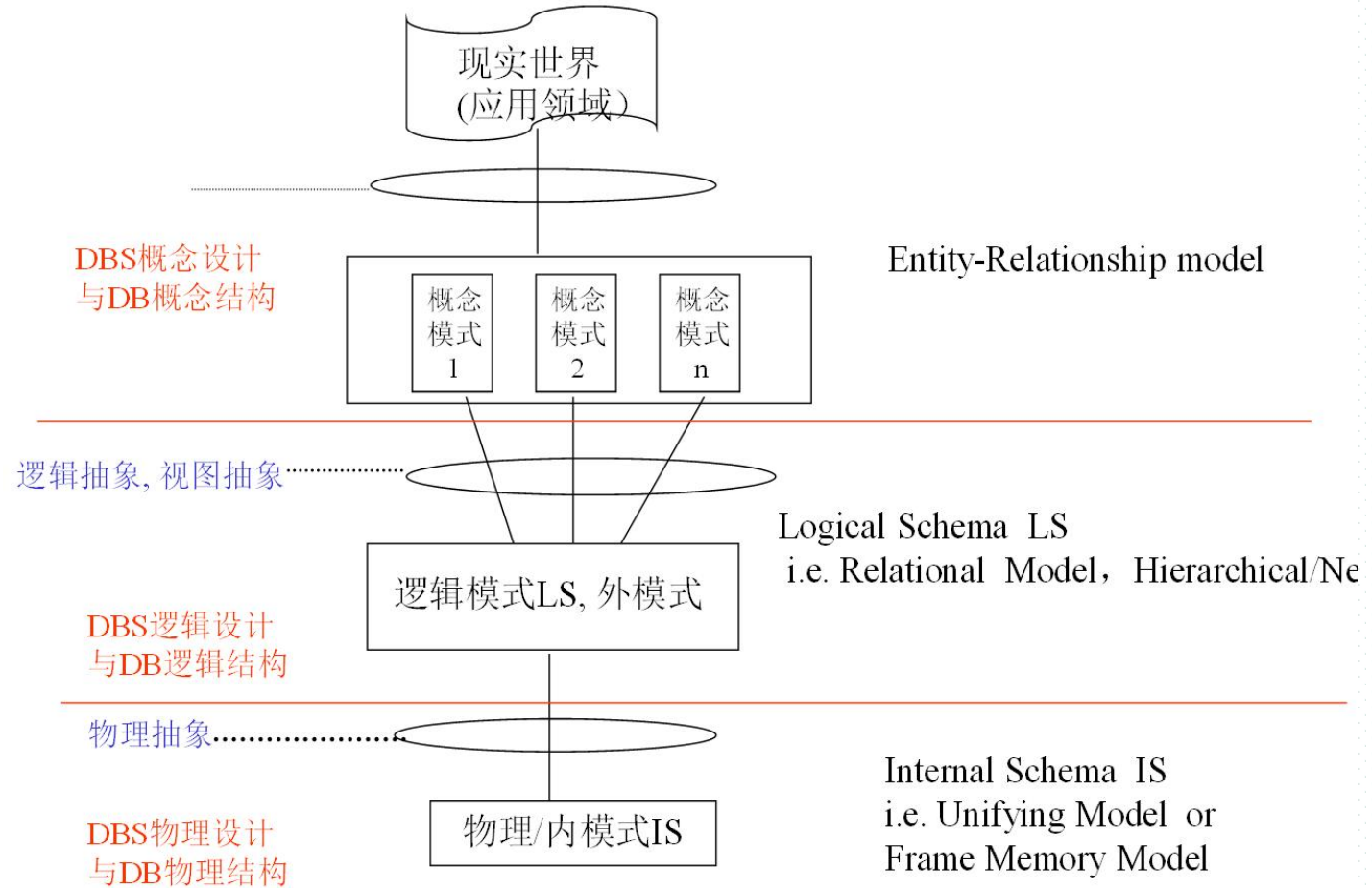


Fig. 6.0.3 DBAS 生命周期模型

Database Design

- 从保持data independence角度出发, 根据data models所定义的数据规范形式, 在view、logical、 physical三个层次, 通过data abstraction, 经过概念设计、逻辑设计、物理设计三个步骤, 构造 面向具体应用领域的DBS的 E-R diagrams, external schema/logical schema , internal schema的集合, 得到conceptual DB、logical DB、 physical DB设计结果
- As described in Fig.1.0.6
 - ▣ conceptual design
 - ▣ logical design
 - ▣ physical design



DBS设计与设计阶段: 数据模型—数据抽象—数据模式

Database Design

■ Logical Design

- ❑ deciding on the database schema. Database design requires that we find a “good” collection of relation schemas.
 - business decision – What attributes should we record in the database?
 - computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?

■ Physical Design

- ❑ deciding on the physical layout of the database
- ❑ depend on DBMS platform

Database Languages

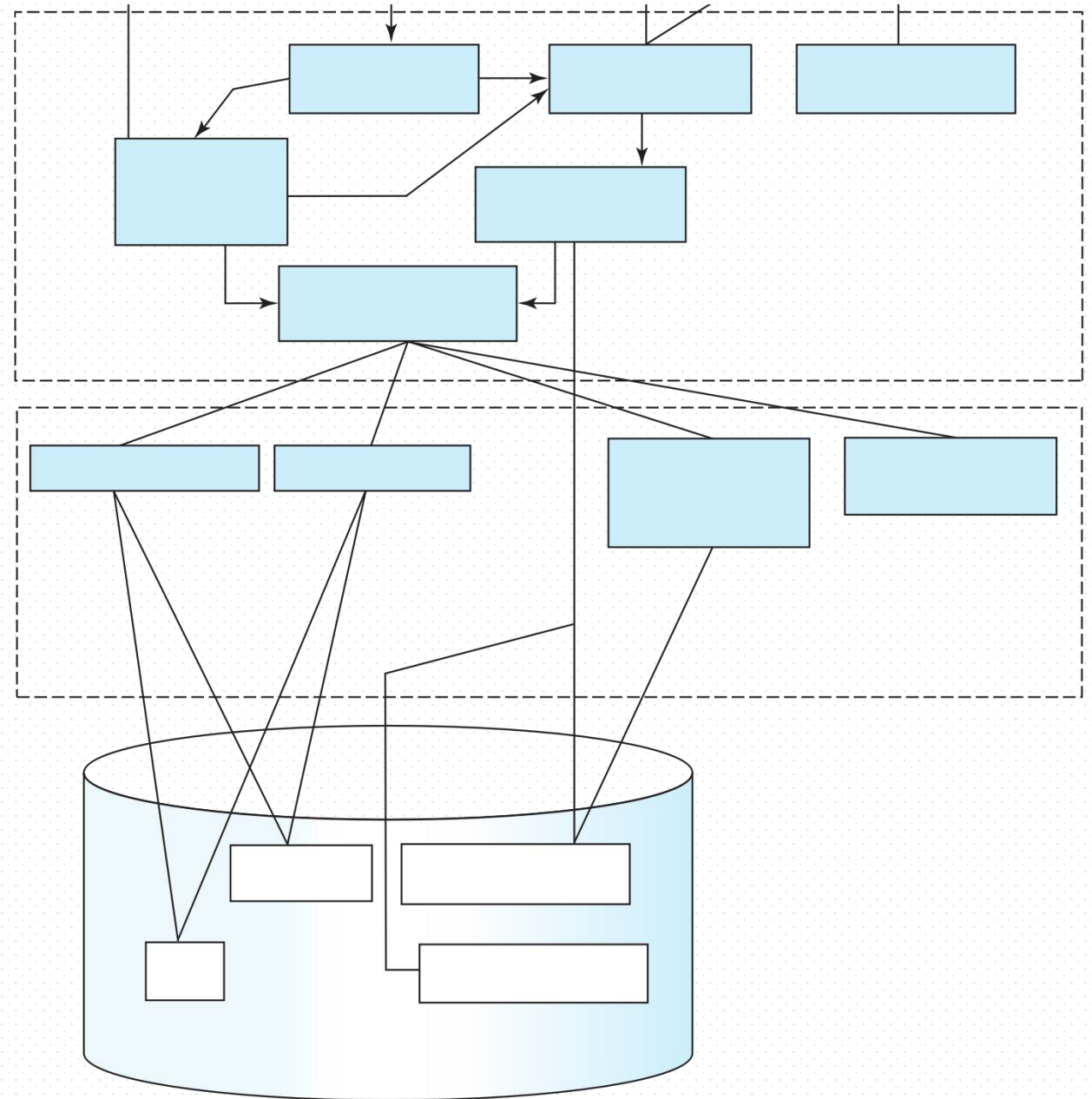
- Database Languages
 - ▣ Data Definition Language (DDL)
 - ▣ Data Manipulation Language (DML)
- Two types of data-manipulation language
 - ▣ **procedural DML** -- require a user to specify what data are needed and how to get those data
 - e.g. relational algebra (关系代数), 元组关系演算, 域关系演算
 - ▣ **declarative DML** -- require a user to specify what data are needed without specifying how to get those data
 - e.g. SQL
- **Data dictionary** contains metadata (i.e., data about data)
 - ▣ Database schema
 - ▣ Integrity constraints
 - Primary key (ID uniquely identifies instructors)
 - ▣ Authorization
 - Who can access what

Database Languages

- Application programs generally access databases through one of
 - ▣ Language extensions to allow embedded SQL (嵌入式SQL)
 - ▣ application program **interface** (e.g., ODBC/JDBC) which allow SQL queries to be sent to a database, i.e. dynamic SQL(动态SQL)

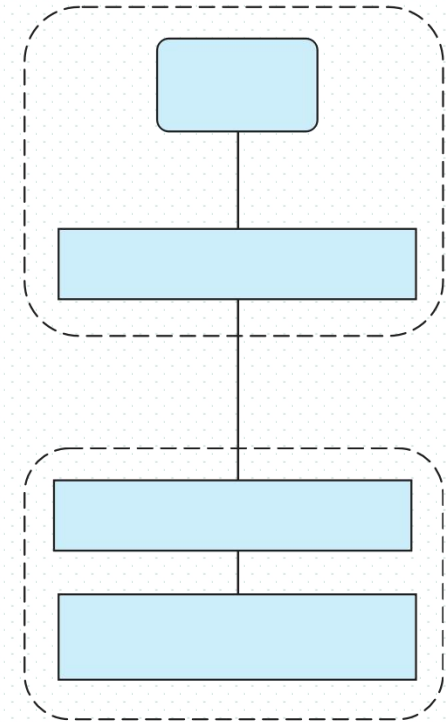
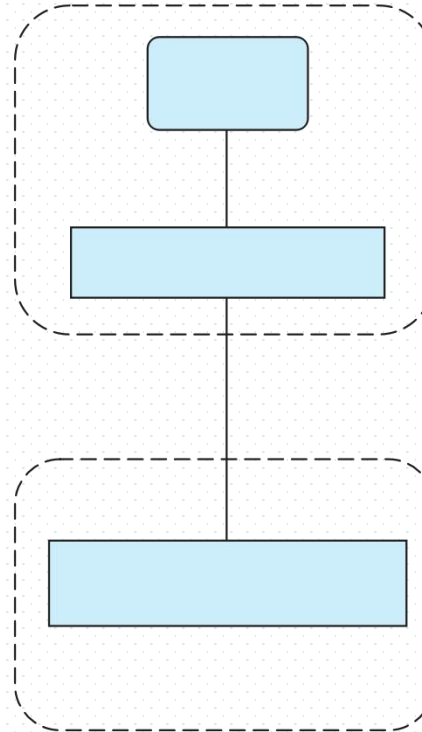
Database Engine/DBMS

- a database system can be divided into
 - ▣ query processor component
 - ▣ storage manager
 - transaction management component



Database Architecture

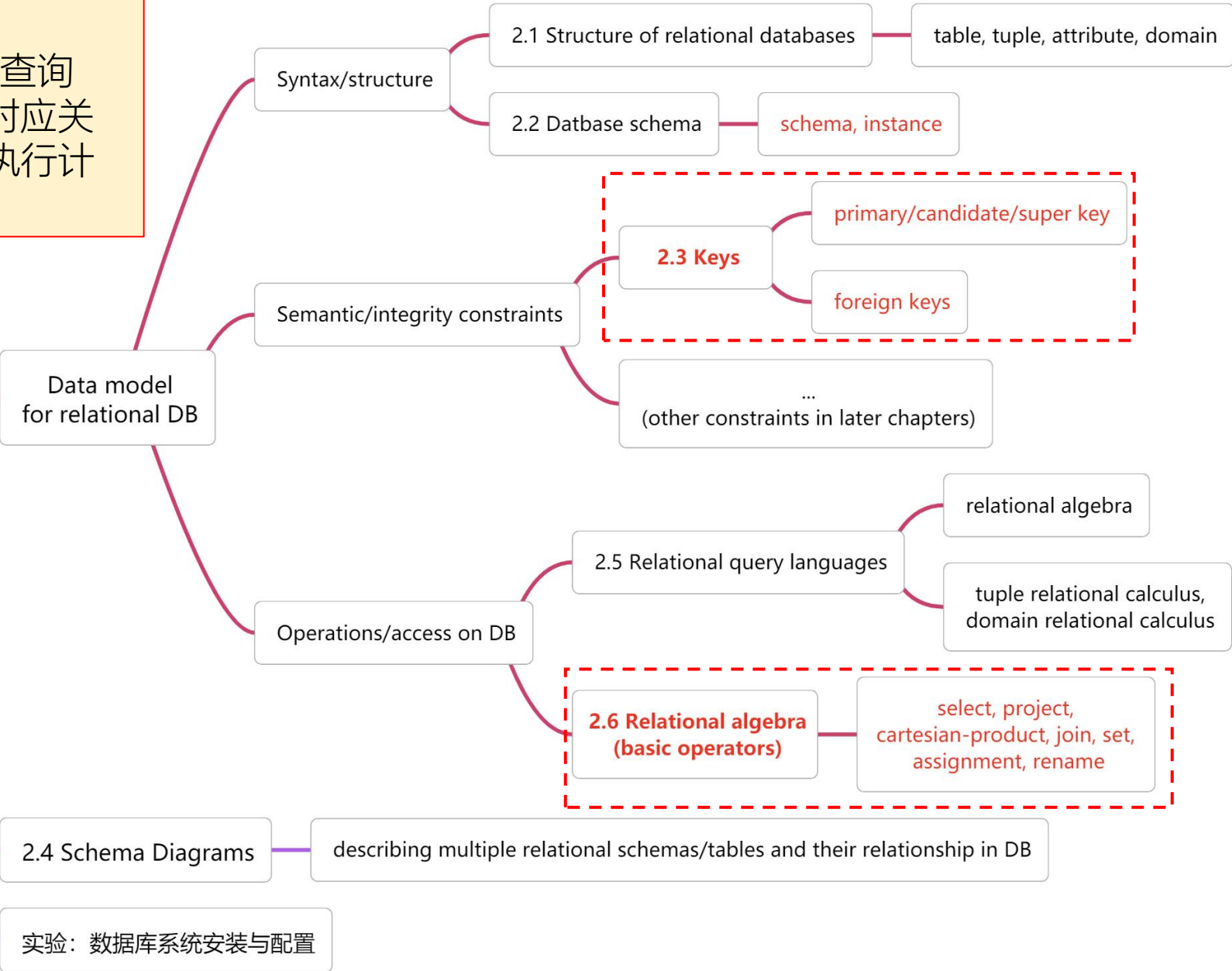
- Database applications are usually partitioned into two or three parts
 - ▣ Two-tier architecture -- the application resides at the client machine, where it invokes database system functionality at the server machine
 - ▣ Three-tier architecture -- the client machine acts as a front end and does not contain any direct database calls.
 - The client end communicates with an application server, usually through a forms interface.
 - The application server in turn communicates with a database system to access data.



Chapter 2 Relational Model

- 要求:
- 1. 用关系代数表达具体查询
 - 2. 关系代数与SQL语句对应关系, 看懂SQL语句查询执行计划(Chapter15-16)

Chapter 2 Introduction to the Relational Model



■ Relational *data model*

- data structure (*syntax*)
- integrity constraints (*semantic*)
 - constraints on schema attributes, e.g. value domain, type, key
 - constraints on dependencies among attributes of a schema, e.g. *age* vs *birth-date*
 - constraints on dependencies among attributes of different schemas, normal forms
- operations/operators on the model, i.e.
 - relational algebra
 - tuple relational calculus
 - domain relational calculus

Relational Algebra

- A procedural language consisting of a set of operations/operator that take one or two relations as input and produce a new relation as their result.
- Basic operators
 - select: σ
 - project: Π
 - **join**, is not a basic operator
 - Cartesian product: $\times$
 - union: $\cup$
 - set difference: $-$
 - rename: ρ

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{\text{salary} \geq 85000}(\text{instructor})$
	Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{ID, salary}(\text{instructor})$
	Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
$\bowtie$ (Natural join)	$\text{instructor} \bowtie \text{department}$
	Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
$\times$ (Cartesian product)	$\text{instructor} \times \text{department}$
	Output all pairs of rows from the two input relations (regardless of whether or not they have the same values on common attributes)
$\cup$ (Union)	$\Pi_{name}(\text{instructor}) \cup \Pi_{name}(\text{student})$
	Output the union of tuples from the two input relations.

Relational algebra expression

■ Relational algebra expression E

- a basic expression is
 - a relation r in the database, or
 - a constant relation $\mathbf{C} = \{c_1, c_2, \dots, c_n\}$, which is written by listing its tuples in the relation
- a general expression is defined as:
 - if E_1 and E_2 are relational algebra expressions, then
 - $E_1 \cup E_2$
 - $E_1 - E_2$
 - $E_1 \times E_2$
 - $\sigma_p(E_1)$
 - $\Pi_s(E_1)$,
 - $\rho_x(E_1)$

, are all relational algebra expressions

SQL statement $\leftrightarrow$ relational algebra expression

Chapter 3-4-5 SQL

■ SQL基本功能

query	Select
Data manipulation	Insert, Delete, Update
Data definition	Create, Drop, Alter
Data control	Grant, Revoke

- 利用SQL DML写出各类查询、插入、更新、删除语句；用DDL定义、修改数据库结构

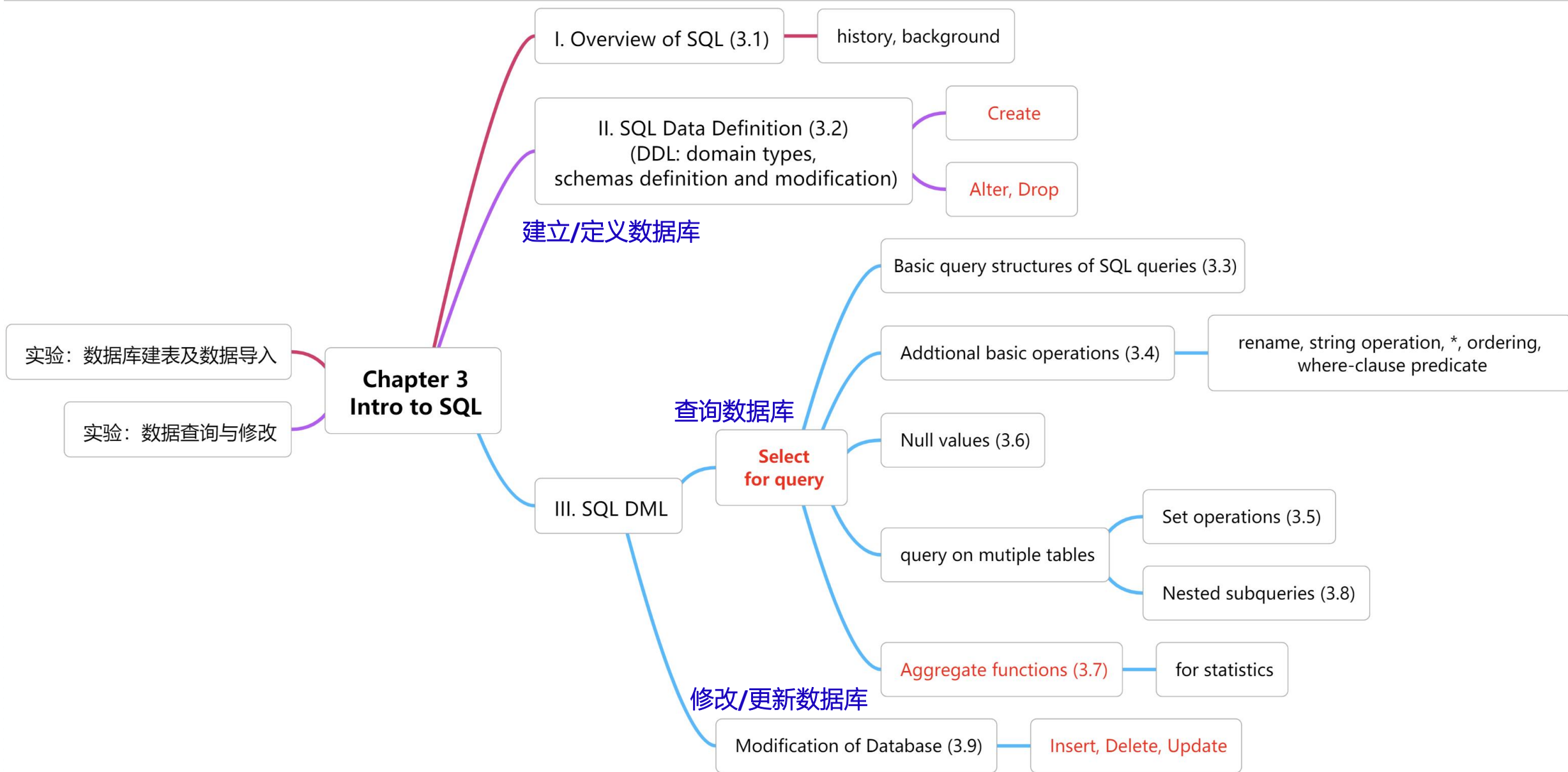
- ▣ 课程实验

- **Select**与关系代数表达式的等价性

SQL statement $\leftrightarrow$ relational algebra expression

- ▣ 关系代数操作/表达式与基本SQL语句的对应关系
- ▣ 面向具体应用领域，写各种查询语句，转换成关系代数表达式

Outline



要求:

1. Create完整性约束定义
2. 参照完整性
3. 多表连接访问

Chapter 4 Intermediate SQL

I. SQL Data types and schemas (4.5)

date and time types

type conversion and
formatting functions

default values

large-object types,
user-defined types

~~create table extensions,
generating unique key values~~

schemas, catalogs, environments

II. 4.1 Join expressions (for multiple table access)

natural join

join conditions

outer joins

join types
and conditions

III. Views (4.2)

definition, query, update

materialized views

IV. Transactions (4.3)

definition, example

VI. Index definition in SQL (4.6)

VII. Authorization (4.7)

grant/revoke, role

实验: 完整性约束

V. Integrity constraints (4.4)

not null, unique, check

referential integrity

complex check constraints and assertions

~~assign names to constraints,
constraints violation during transactions~~

■ Integrity constraint

- entity/domain integrity/**primary key**
- referential integrity
 - **foreign key**, cascade delete/update
- **check**, not null
- functional dependency, $X \rightarrow Y$

利用SQL语句,
定义、判断各类约束

■ Integrity schemes

- 利用*create* in SQL, *check*, 定义完整性约束
- 利用**Select**语句表示和检查主键、外键、函数依赖
- 实验, 作业, 期中测验
- ~~【略 assertions, triggers】~~

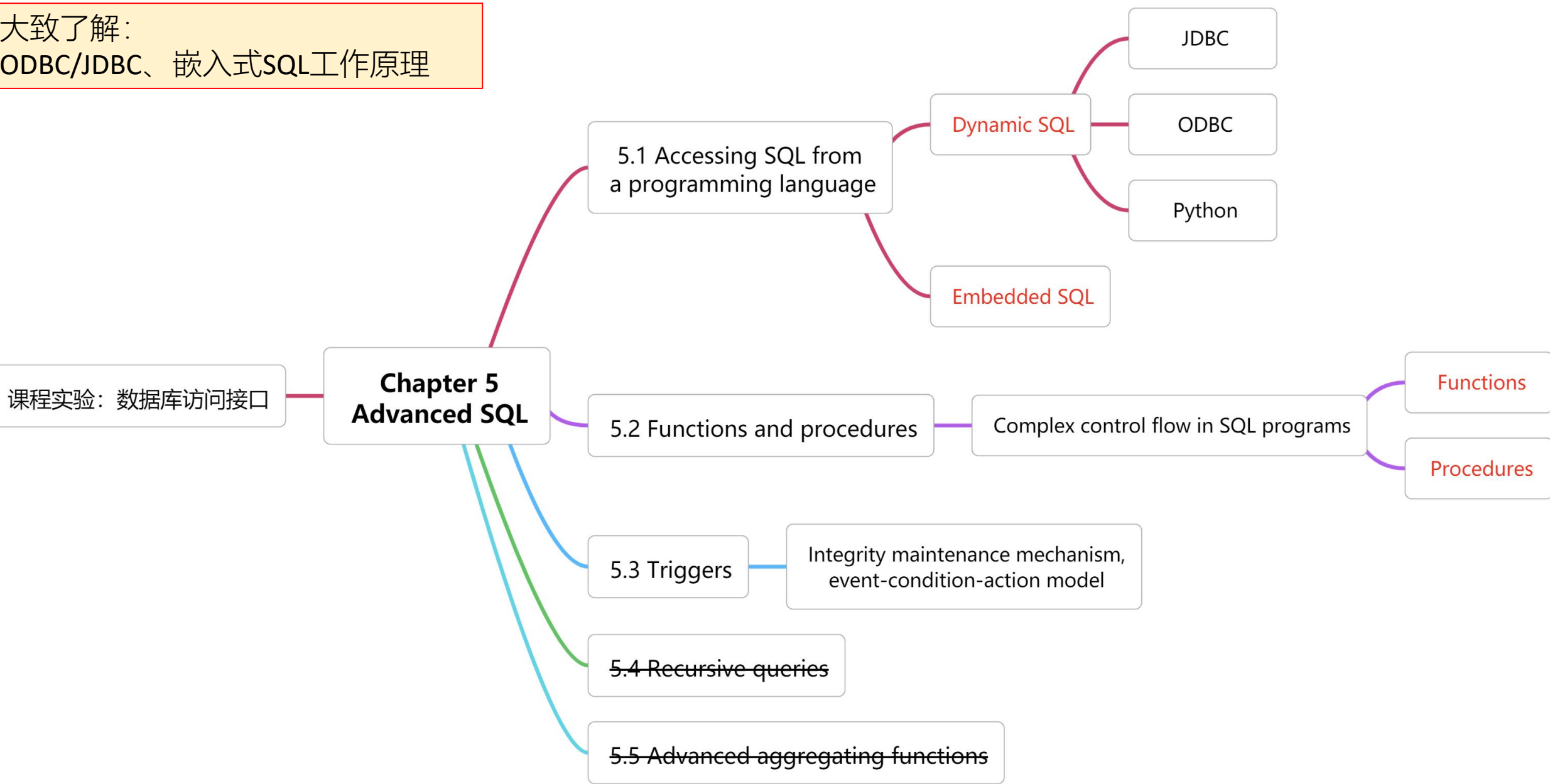
■ 连接操作与多表查询

■ Security and Authorization

- 基本概念, *grant*, *revoke*语句

Outline

大致了解：
ODBC/JDBC、嵌入式SQL工作原理



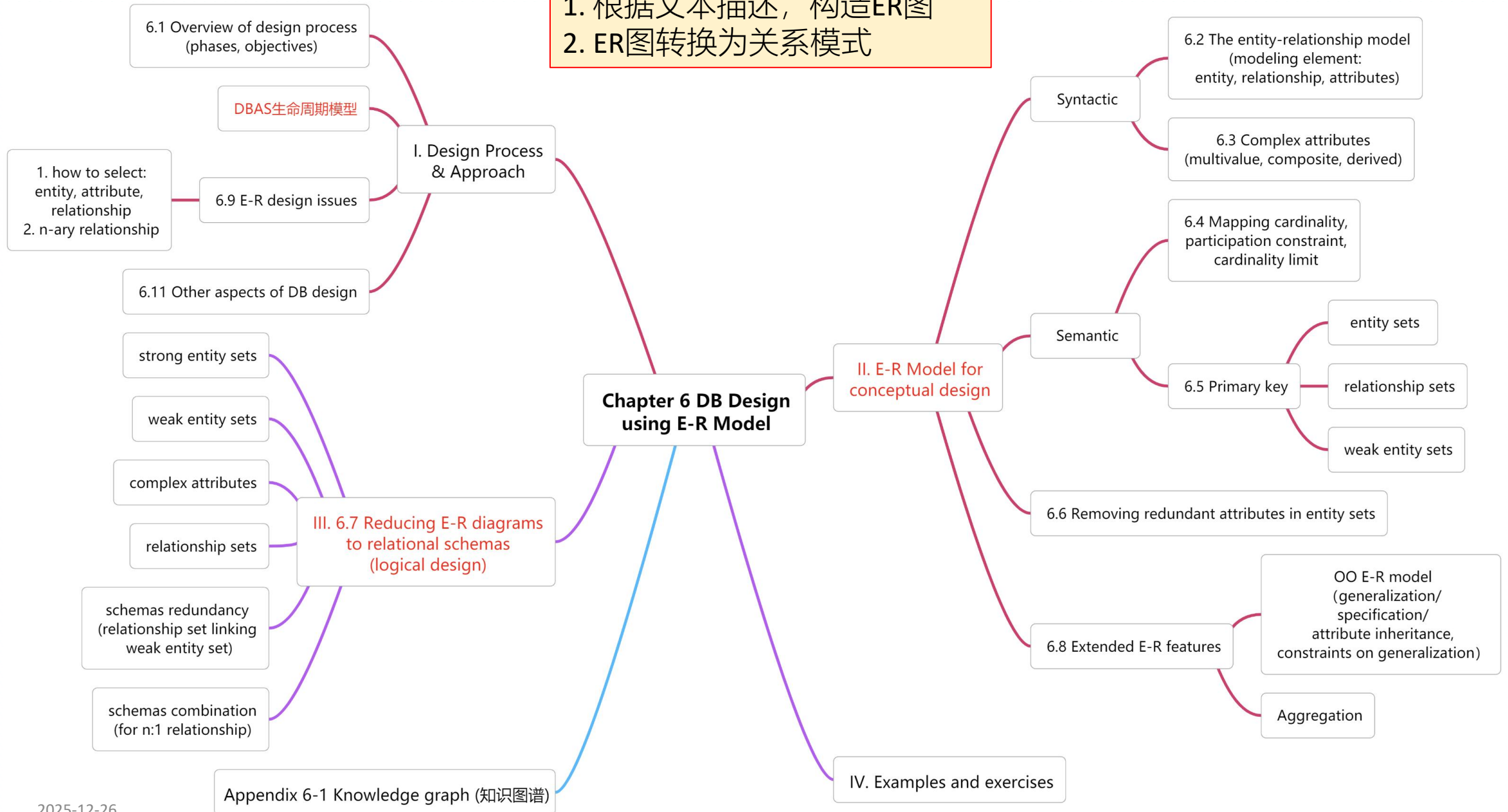
Chapter 6 Database Design Using the E-R Model

- E-R model
 - entity set, relationship set
 - ternary relationship, 转为二元联系+特殊实体集
 - attribute
 - composite, multivalued, derived
 - constraints
 - cardinality constraint, participation constraint
 - cardinality limits, 根据cardinality limits推断cardinality constraint和participation constraint
 - key
 - weak entity sets
- Extended E-R model
 - superclass, subclass, IS-A联系, 继承inheritance
 - 聚集

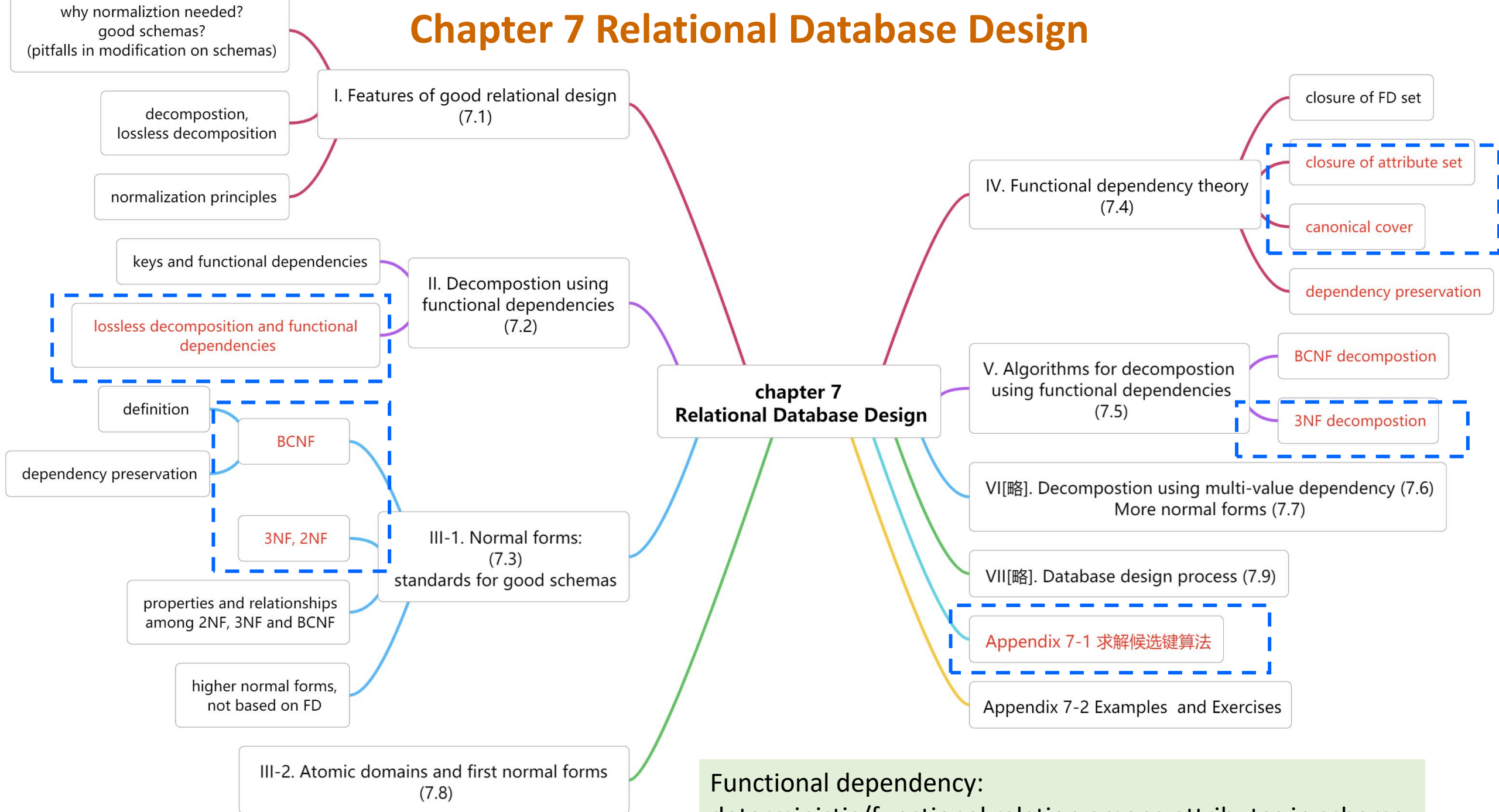
Database Design Using the E-R Model

- Reducing E-R Diagrams to relation schemas
 - entity set, relationship set → relational tables
- 注意
 - composite attribute, e.g. <firstname, lastname>: 子属性转为表中单独属性
 - multivalued attribute: 转为单独表, 表中属性包括: 实体集主键+多值属性
 - many-to-many relationship set, 单独转表
 - many-to-one relationship sets with partial participation on the many-side, 联系可以单独转表, 也可以不单独转表
 - many-to-one relationship sets with total participation on the many-side, 联系不单独转表, 归入many-side的表中
 - 多元联系→可直接转为关系表, 表中属性包括联系所关联的实体集的主键
 - weak entity sets: 所依赖的强实体集的主键+弱实体集属性→ relational table
 - ISA联系转表: 父类实体主键+子类实体全部属性
- 例题与作业
 - 面向具体应用领域, 设计E-R图 + 将E-R图转换成关系表

要求:
1. 根据文本描述, 构造ER图
2. ER图转换为关系模式



Chapter 7 Relational Database Design



Functional dependency:
deterministic/functional relation among attributes in schema

Normalization Principles

- 在逻辑DBS设计过程中，将E-R图转换，得到面向特定应用领域的初始关系模式集
- 这些初始关系模式集中可能存在多种（作为完整性约束的）关系模式属性间的**数据依赖** (Data Dependencies) 关系
 - ▣ **函数依赖** (functional dependencies, FD, §7.2)
 - ▣ **多值依赖** (Multivalued Dependencies, MVD, §7.4, 7.6)
 - ▣ **连接依赖** (Join Dependencies, JD) 略
- 直接根据初始关系模式构造DBS，由于初始关系模式中**数据依赖**关系的存在，**可能会**违反DB的完整性约束，导致DBS使用过程中出现如下问题，影响DBS的正确性、性能、效率
 - ▣ 数据冗余问题、插入问题、更新问题、删除问题(pitfalls, §7.1)

Normalization Principles

- 因此，对初始关系模式集，需要根据关系规范化理论，在保证关系模式的

- 函数无损连接性 (lossless join), 和/或
- 函数依赖保持性 (dependency preservation)

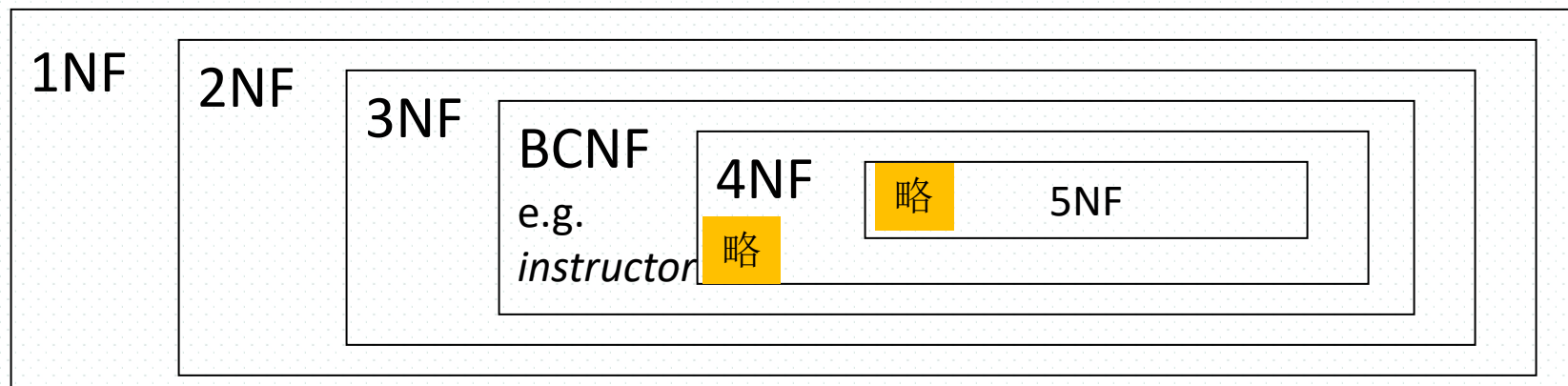
约束前提下，对关系模式集进行规范化处理——等价变换/模式分解

- 关系模式规范化主要步骤为

- 根据函数依赖的Armstrong's 公理系统 (§7.4.1) 和多值依赖的公理系统，从初始关系模式集中已知的函数依赖和多值依赖出发，推导出初始关系模式集中所有的函数依赖 (§7.4) 和多值依赖
- 对具有函数依赖和多值依赖的初始关系模式集，采用
 - 模式分解算法，对其进行（等价）分解和变换，将其转换为各种范式形式，包括：
 - 1NF (§7.8)、2NF (Exercise)、BCNF (§7.3.1)、3NF (§7.3.2)、4NF (§7.6)、5NF (§7.7), 以消除模式集中的函数依赖和多值依赖带来的负面影响, 保证数据库完整性

Normalization Principles

- 关系模式规范化处理的基本要求为:
 - 静态关系具有第一范式形式
 - (理论上) 动态关系最好具有3NF或BCNF形式
- 3种数据依赖间的关系
 - 函数依赖是特殊的多值依赖
 - 多值依赖又是连接依赖的特例
- 范式1NF、2NF、3NF、BCNF可以看作由符合范式要求的各种关系模式组成的关系模式的集合
 - e.g. $1NF = \{ R \mid R \text{满足第一范式的定义} \}$
- 各种范式间的关系, 参见Fig.7.0.3



要求：范式定义，范式特点

函数依赖保持性

无损连接性

1NF



消除非主属性对键的部分函数依赖

2NF

[特点]: 不存在非主属性对键的部分依赖, 非主属性对键完全依赖



消除非主属性对键的传递函数依赖

3NF

[特点: 不存在非主属性对键的部分、传递依赖]



消除主属性对键的部分和传递函数依赖

BCNF

[不存在非主属性、主属性对键的部分、传递依赖]



消除非平凡且非函数依赖的多值依赖

4NF



消除非平凡连接依赖

5NF

Fig. 7.0.3 关系范式特点及间相互关系

Normalization Principles

- 给定一个关系模式，可以采用规范化算法将其转换为1NF、2NF、3NF、BCNF
- 对连接依赖和第五范式，无相应的模式规范化算法

习题类型

- 给定关系表 $r(R)$ 和若干函数依赖 $\alpha \rightarrow \beta$, 判断 r 是否满足 $\alpha \rightarrow \beta$
- 根据Amstron公理系统, 判断关于函数依赖的一些公式是否成立
 - 例如, 如果 $A \rightarrow B, B \rightarrow C$, 则 $A \rightarrow C$
- 特殊函数依赖的判断
 - 传递、部分函数依赖
- 根据文字描述, 抽象出函数依赖关系
- 设计SQL语句, 判断函数依赖和主键、外键
- 利用算法, 求候选键 (Appendix 7-1)
- 计算属性闭包 α^+
- 计算函数依赖集 F 的最小正则集 F_c

习题类型

- 给定关系模式R和定义在R上的函数依赖集F，根据定义，判断R属于第几范式，为什么？
 - ▣ 只考虑[1NF,]2NF, 3NF, BCNF
- 判断一个模式分解是否为无损连接、~~函数依赖保持~~（定义）
 - ▣ 判断无损连接的定理
- 给定非3NF的关系模式R和定义在R上的函数依赖集F，将R分解为第三范式
 - ▣ 1. 求候选键 2. 求Fc 3. 分解
- ~~给定非BCNF的关系模式R和定义在R上的函数依赖集F，将R分解为BCNF范式~~

Chapter 13 Data Storage Structures

【了解相关概念即可】

■ File Organization

- a database is stored as a collection of **DB files**,
 - e.g. SQL Server, 主文件, 辅文件, 日志文件
- each file is a sequence of *records*, and a record is a sequence of *fields*
- 数据库表中的元组tuple作为记录record存储在DB文件中
 - a *tuple* is stored as a *record* in the DB file
- record in files
 - fixed-length records
 - variable-length records, e.g. *name* varchar(20)

Data Storage Structures

■ Organization of Records in Files

DB file can be viewed as a set of **records**, these records are logically organized in one of the following ways

□ **Heap** File Organization

- 无主键、无索引的关系表组织为heap文件
- e.g. 向表中连续插入多个元组, 组织为堆文件

□ **Sequential** File Organization

- 有主键、索引的关系表组织为顺序文件, e.g. B⁺-tree on instructor

□ **Multitable Clustering** File Organization

- 来自多张表的多种类型记录组织在一个文件中, *department* ⋈ *instructor*

□ **Hashing** File Organization

- the file records are stored in a number of **buckets**, the #bucket is the address of the record
- a hash function on some attributes of file records (i.e. search key) is used to determine the addresses of the records in the file

Data Dictionary Storage

Data dictionary (also called system catalog) stores metadata; that is, data about data, such as

- Information about relations
 - ▣ names of relations
 - ▣ names, types and lengths of attributes of each relation
 - ▣ names and definitions of views
 - ▣ integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data ✕
 - ▣ e.g. number of tuples in each relation
 - ▣ used for query optimization, as described in Chapter 15/16
- Physical file organization information
 - ▣ how relation is stored (sequential/hash/...)
 - ▣ physical location of relation
- Information about indices (Chapter 14)

Chapter 14 Indexing

■ 概念

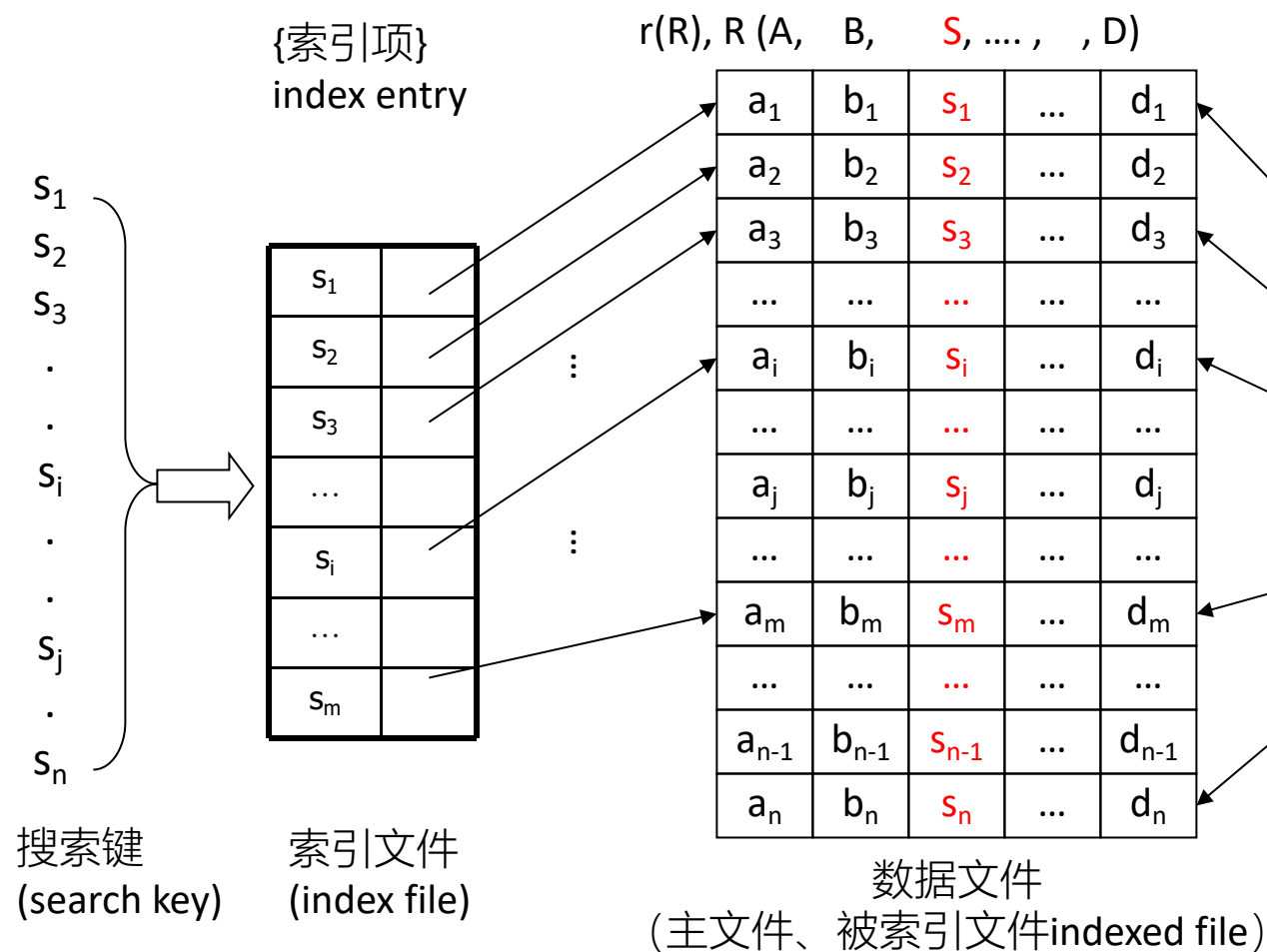
- order index, hash index
 - 有序索引支持等值、比较、范围查询
 - Hash索引只支持等值查询
 - dense/sparse indices
 - primary/secondary indices, clustering/non-clustering indices
- [各类索引特点, 判断索引类型]

■ 【略 B/B+ tree

——了解基本概念、结构特点

- ~~平衡树, 多叉树~~
- ~~叶节点稠密, 非叶节点稀疏】~~

ordered indices



hash indices

$h(s_1)$

$h(s_i) \rightarrow \#buket$

$h(s_i)$

$\vdots$

$h(s_n)$

散列
函数H

搜索键
(search key)

图 14.0.1 索引技术(indexing)及其分类

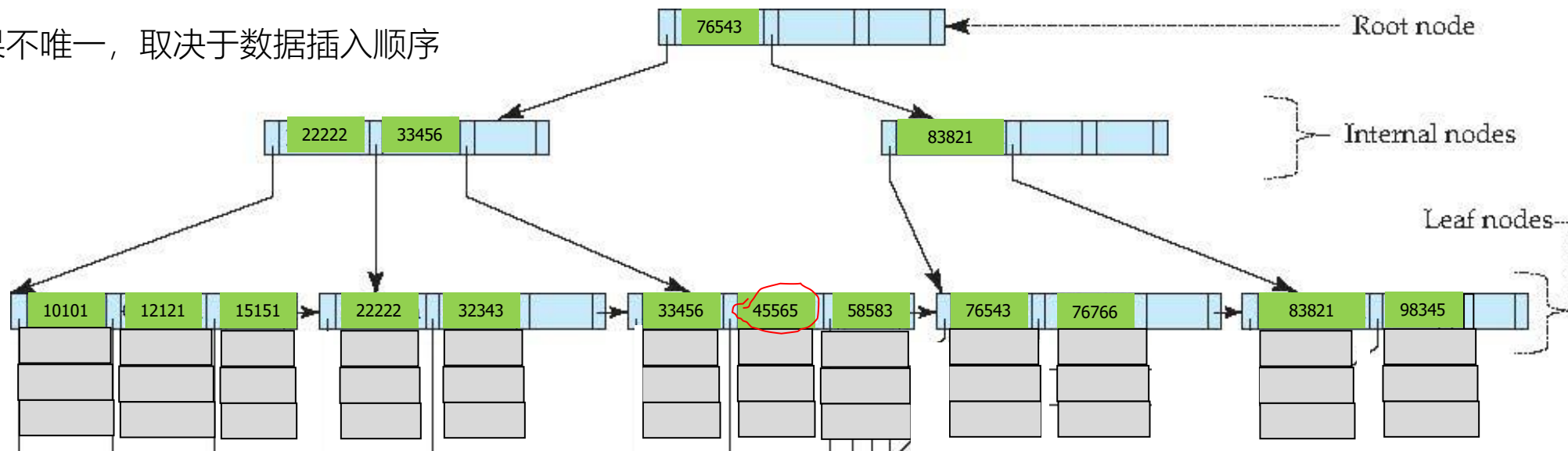
B+-Tree (n=4) Index Sequential File

n: degree or branching factor

文件记录直接
存储在叶节点

Primary/Clustered index on ID

B+树结果不唯一，取决于数据插入顺序



```
create table instructor
  (ID integer, primary key;
   ....
  )
```

Or:

```
create clustered index IDIndex
  on instructor(ID)
```

支持:

1. 等值查询, e.g. ID=10101
2. 范围查询, e.g. ID between 10101 and 32323

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

支持:

1. 等值查询, e.g. name='Gold'
2. 范围查询,
e.g.name between 'Gold' and 'Wu'

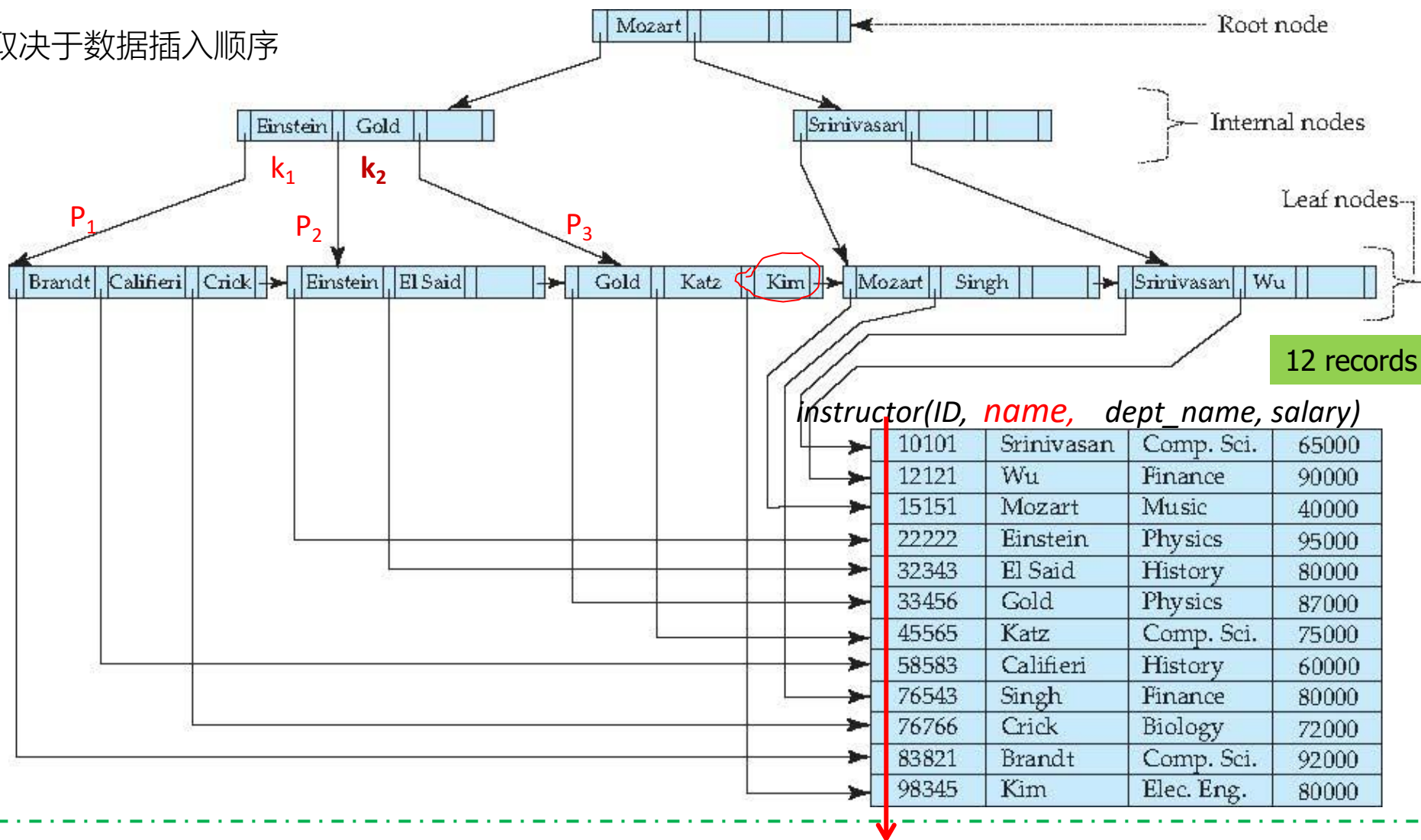
B⁺-Tree ($n=4$)

secondary/nonclustered index on *name*

n : degree or
branching factor

叶节点存储指向文件记录的指针

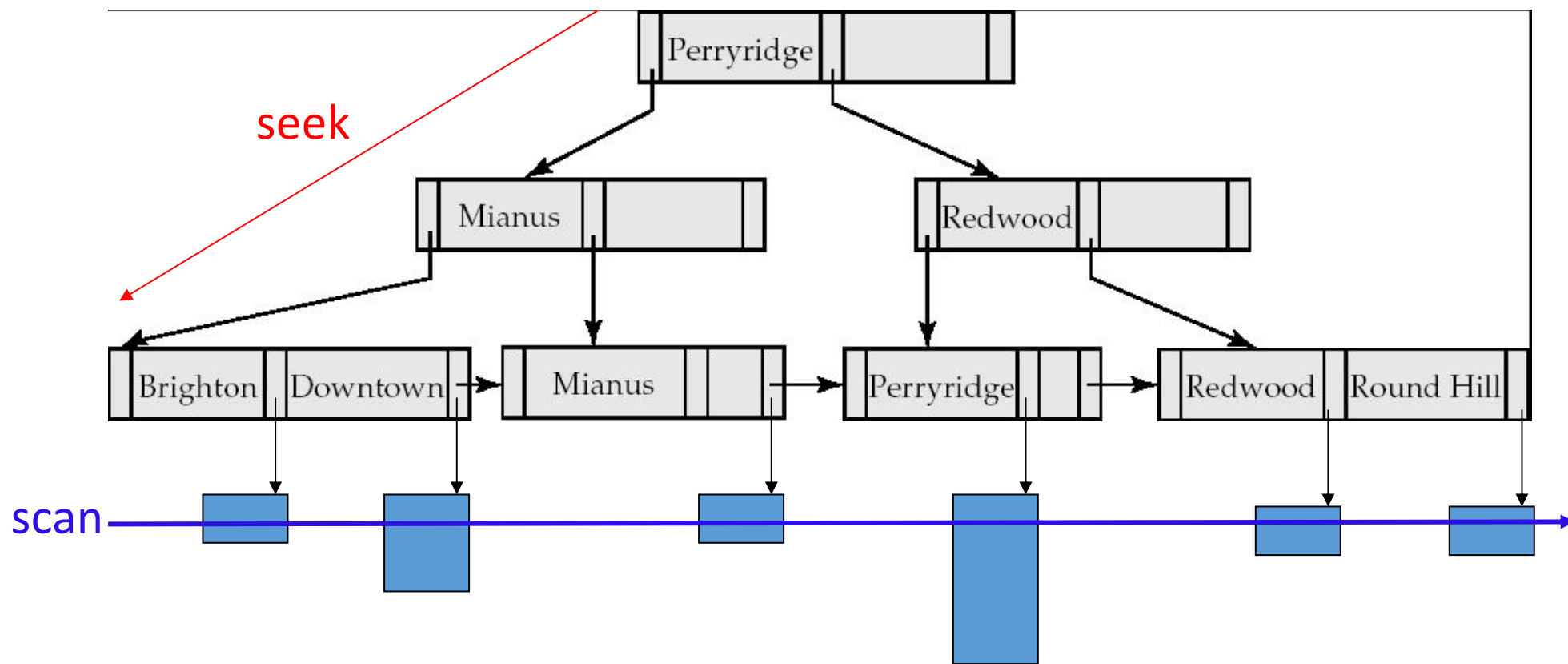
B⁺树结果不唯一, 取决于数据插入顺序



聚集索引树：索引+数据

访问方式：

1. 聚集索引查找seek, e.g. where branch-name='Brightton', 二分查找, 快速
2. 聚集索引扫描scan, e.g. where account-number=10485, 线性扫描



■ 例题与作业

□ SQL create index创建索引

- 区分聚集/非聚集索引索引的定义

create clustered/nonclustered index CIndexName on Student (ID)

□ 针对特定SQL语句， e.g. select/update/insert/delete， 分析如何在表的属性上建立索引， 判断索引是否达到预期加速查询的效果

□ 复合索引

- 左前缀原则

□ 根据定义判断集/非聚集、稠密/非稠密索引

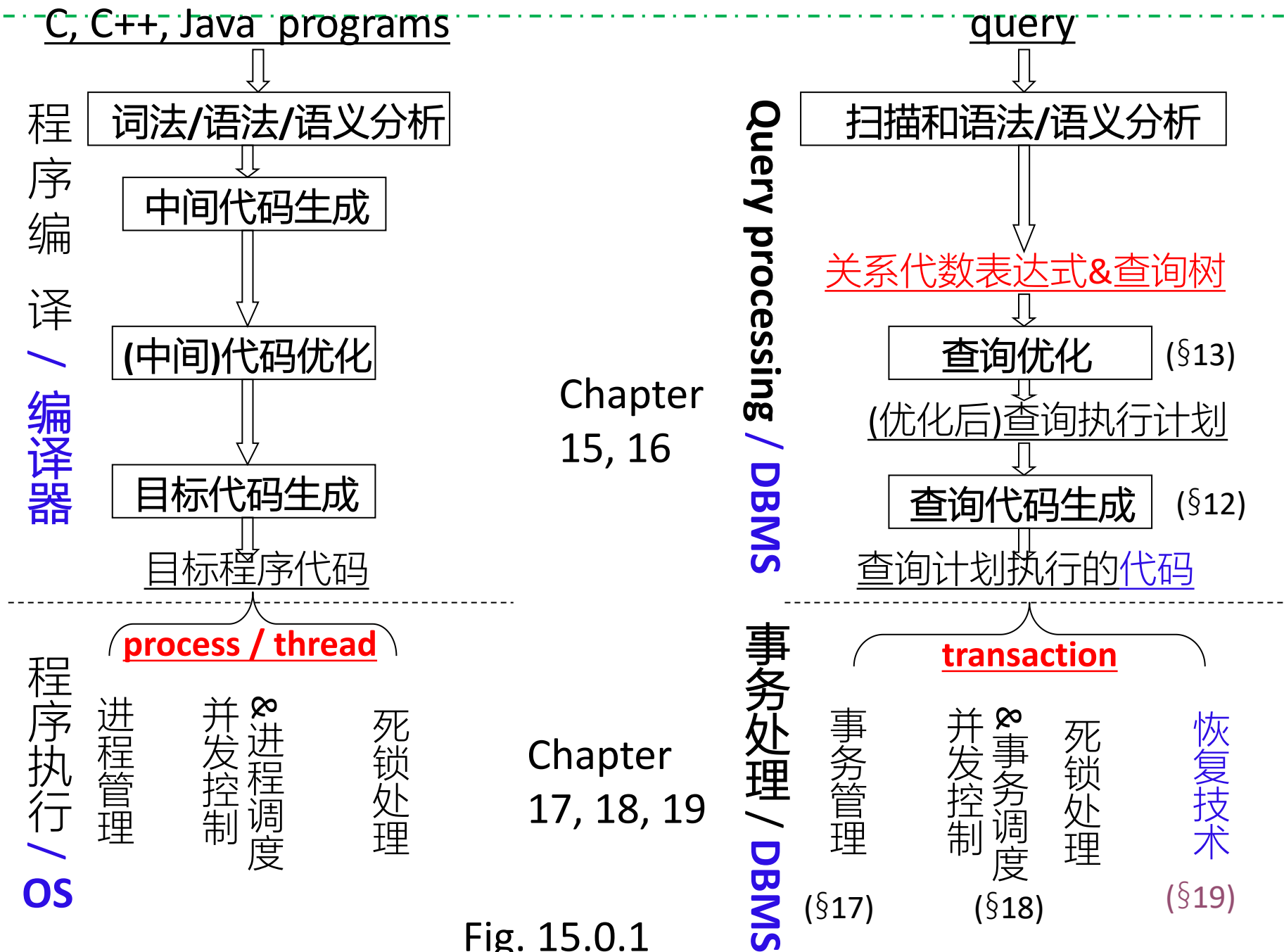
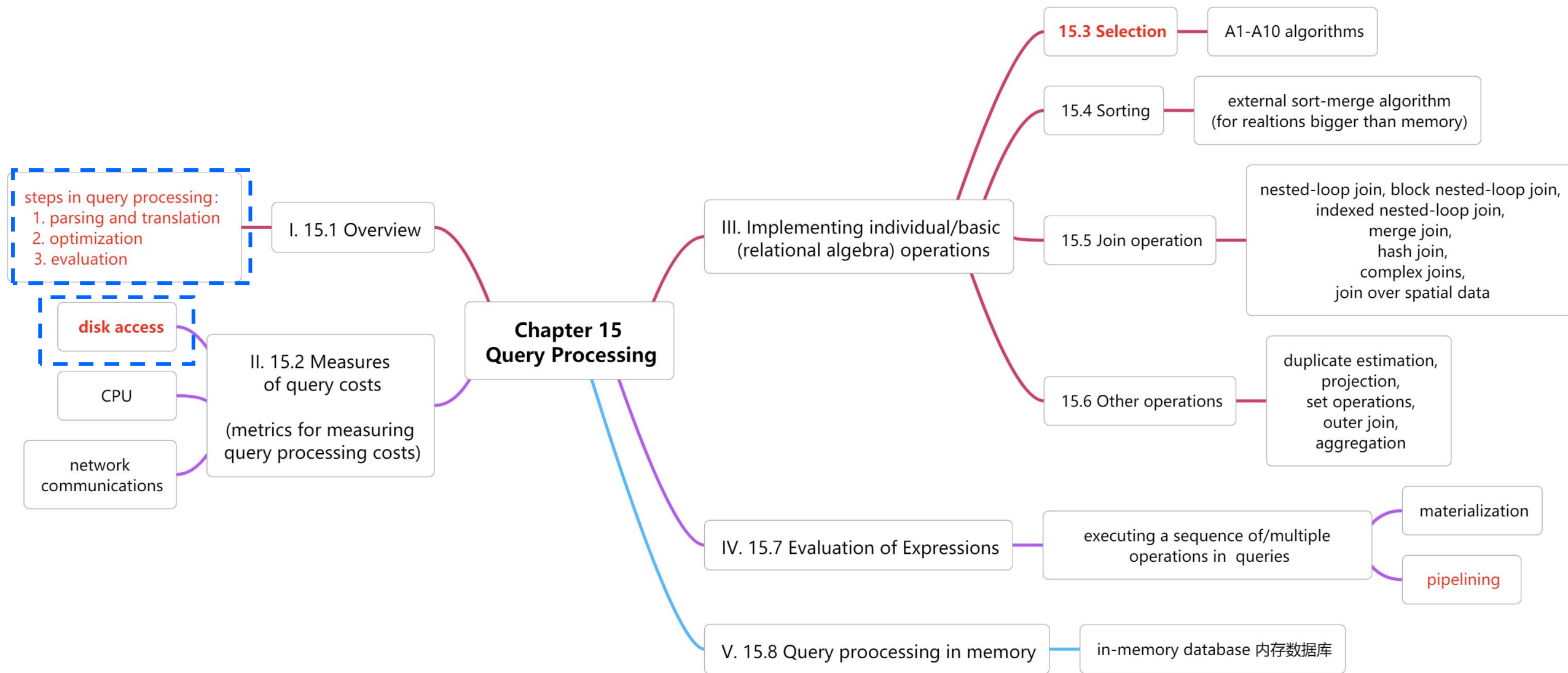


Fig. 15.0.1

Chapter 15 Query Processing



Query Processing Steps

■ Steps in query processing

▣ Fig. 15.1

```
select salary
from instructor
where salary < 2500
```

1. Parsing and translation
2. Optimization
3. Evaluation

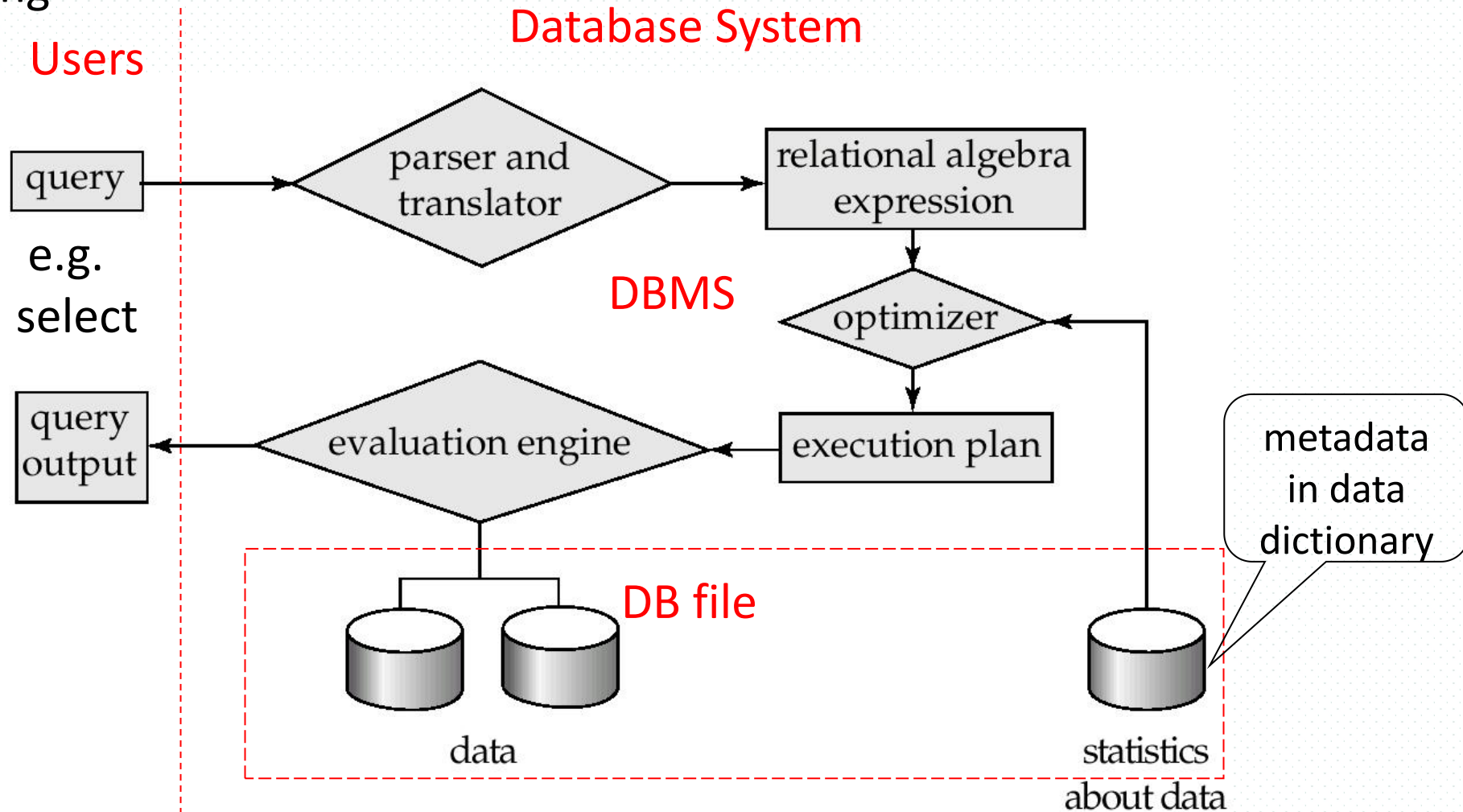
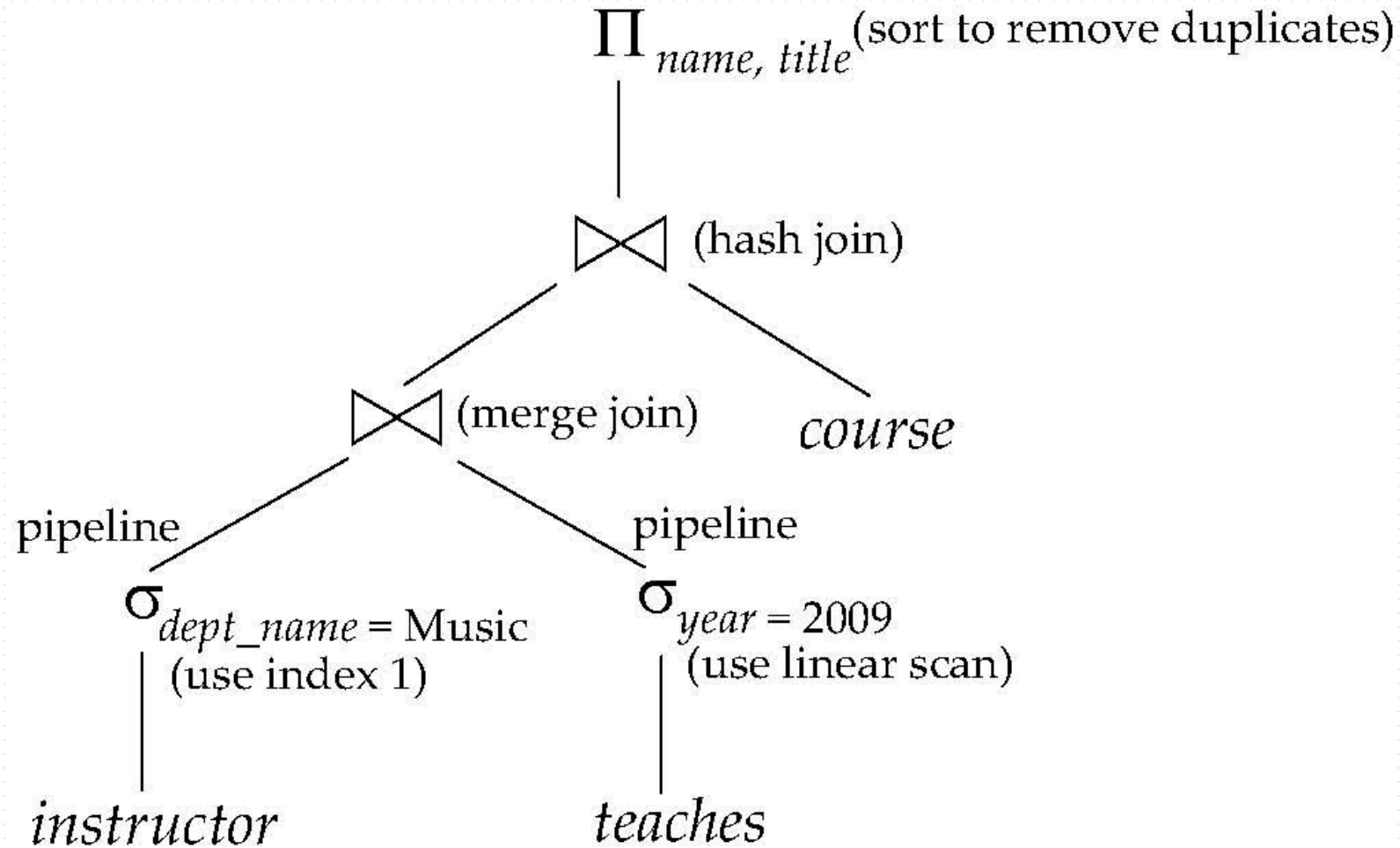


Fig.15.1 Steps in query processing

Query tree vs Query evaluation plan



Measures of Query Costs 【略，大致了解】

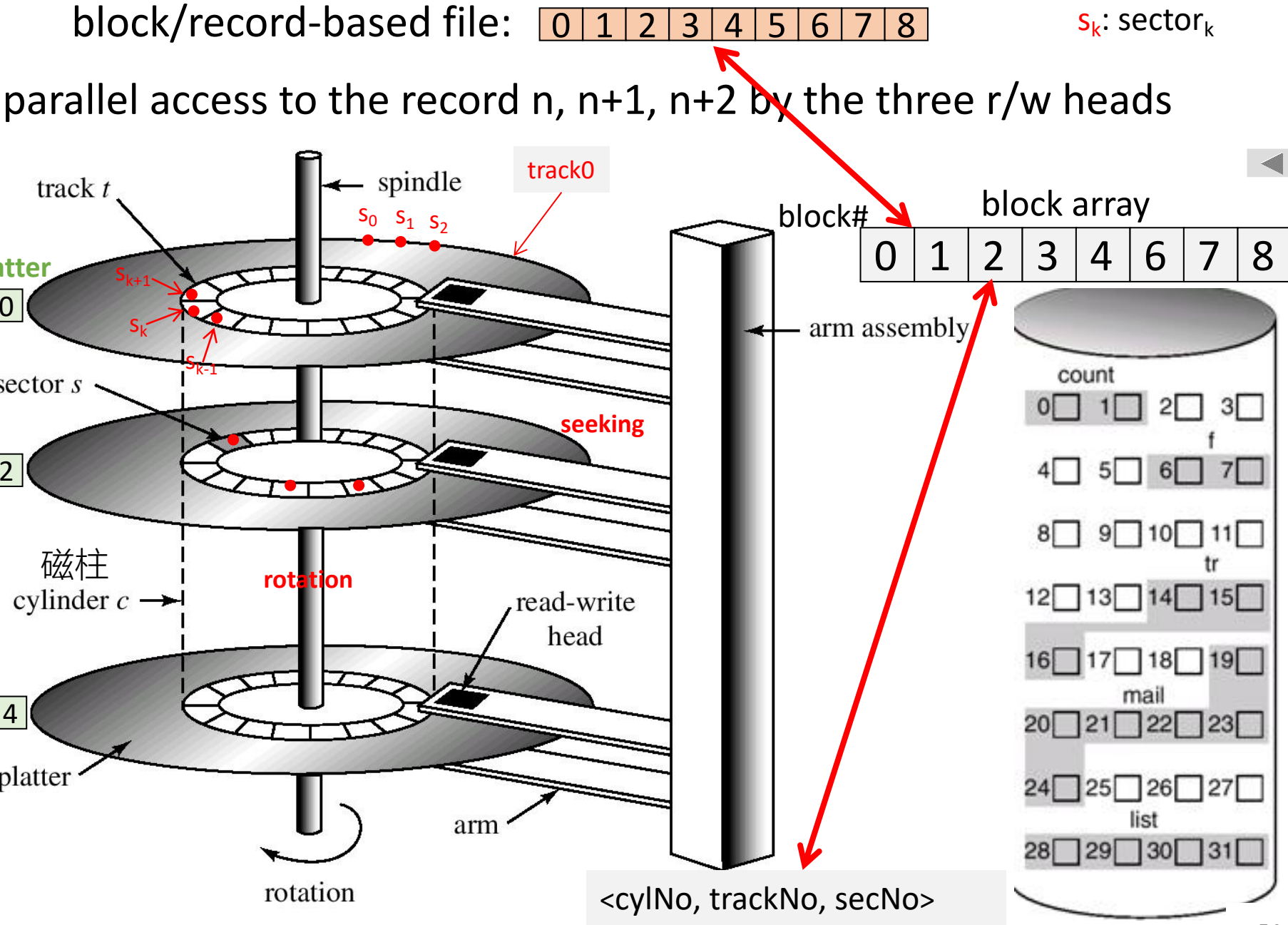
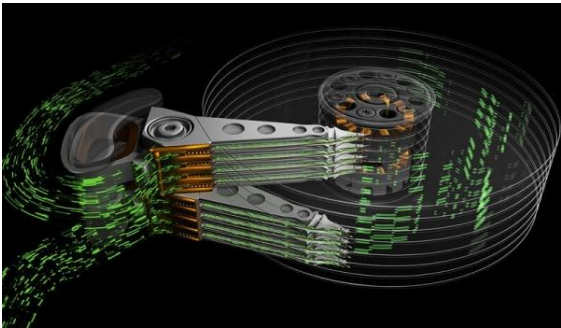
- Cost is generally measured as **total elapsed time** for answering query, and many factors contribute to time cost
 - **disk access**
 - CPU
 - or even network communication
- Disk access is the **predominant** cost
 - number of seeks * average-seek-cost
 - 数据定位时间，需要访问的records在DB file中的位置
 - number of blocks read * average-block-read-cost
 - number of blocks written * average-block-write-cost
 - cost to write a block is greater than cost to read a block
 - data is **read back after** being written to ensure that the write was successful

Fig. 12.1

一维块地址block#

- 三维物理地址 on disk
- 柱面号cylNo
 - 磁道号trackNo
 - i.e. 盘面, 磁头
 - 扇区号secNo

磁头+盘片



■ 内部传输速率

- ❑ 磁头在盘片上读写数据速率，如50MB/s
- ❑ 也称为：（最大最小）持续传输速率
- ❑ 取决于seek time, latency

■ 外部传输速率

- ❑ 硬盘缓存cache（磁盘控制器中的I/O 寄存器）与内存间的传输速率
- ❑ 也称为：突发数据传输速率、接口速率
- ❑ 依磁盘接口类型而定，如SATA, m.2

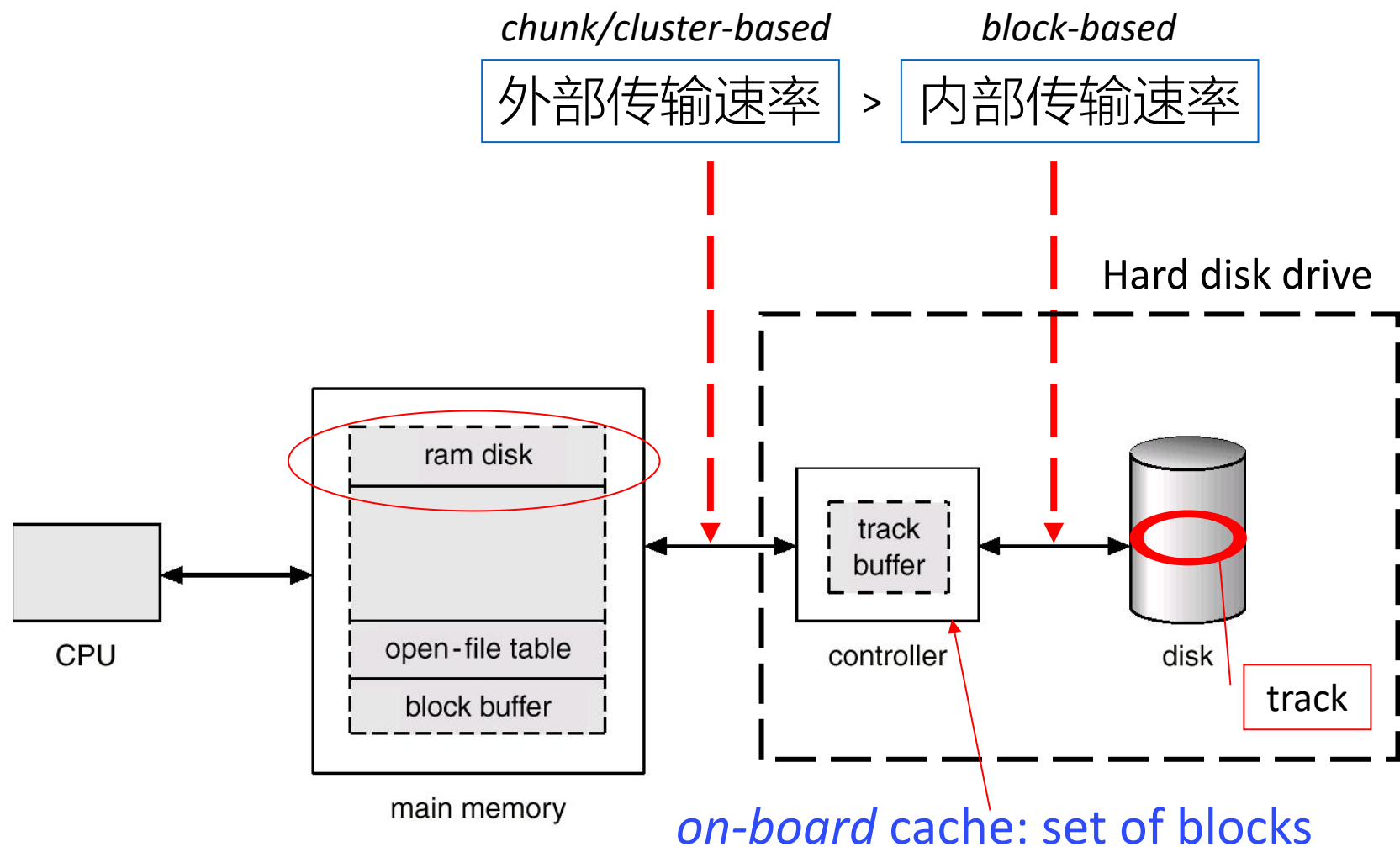


Fig. 12.11.0 on-board cache in device controller

Measures of Query Costs 【略】

- For simplicity we just use the **number of block transferred** *from disk* and the **number of seeks** as the cost measures
 - t_T – time to transfer one block
 - disk的内部传输速率
 - t_S – time for one seek
 - 包括：寻道时间，旋转延迟
 - **time cost** for b block transfers plus S seeks
$$b * t_T + S * t_S$$

$$\text{cost} = \langle b, S \rangle$$

~~select algorithms A1-A10 【略】~~

	Algorithm	Cost	Reason				
A1	Linear Search	$t_S + b_r * t_T$	One initial seek plus b_r block transfers, where b_r denotes the number of blocks in the file.	A4	Secondary B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	This case is similar to clustering index.
A1	Linear Search, Equality on Key	Average case $t_S + (b_r/2) * t_T$	Since at most one record satisfies the condition, scan can be terminated as soon as the required record is found. In the worst case, b_r block transfers are still required.	A4	Secondary B ⁺ -tree Index, Equality on Non-key	$(h_i + n) * (t_T + t_S)$	(Where n is the number of records fetched.) Here, cost of index traversal is the same as for A3, but each record may be on a different block, requiring a seek per record. Cost is potentially very high if n is large.
A2	Clustering B ⁺ -tree Index, Equality on Key	$(h_i + 1) * (t_T + t_S)$	(Where h_i denotes the height of the index.) Index lookup traverses the height of the tree plus one I/O to fetch the record; each of these I/O operations requires a seek and a block transfer.	A5	Clustering B ⁺ -tree Index, Comparison	$h_i * (t_T + t_S) + t_S + b * t_T$	Identical to the case of A3, equality on non-key.
A3	Clustering B ⁺ -tree Index, Equality on Non-key	$h_i * (t_T + t_S) + t_S + b * t_T$	One seek for each level of the tree, one seek for the first block. Here b is the number of blocks containing records with the specified search key, all of which are read. These blocks are leaf blocks assumed to be stored sequentially (since it is a clustering index) and don't require additional seeks.	A6	Secondary B ⁺ -tree Index, Comparison	$(h_i + n) * (t_T + t_S)$	Identical to the case of A4, equality on non-key.

- ~~Join algorithms~~

- Nested-loop join
 - Block nested-loop join
 - Indexed nested-loop join
- Merge-join
- Hash-join

- Two strategies are used for expression evaluation

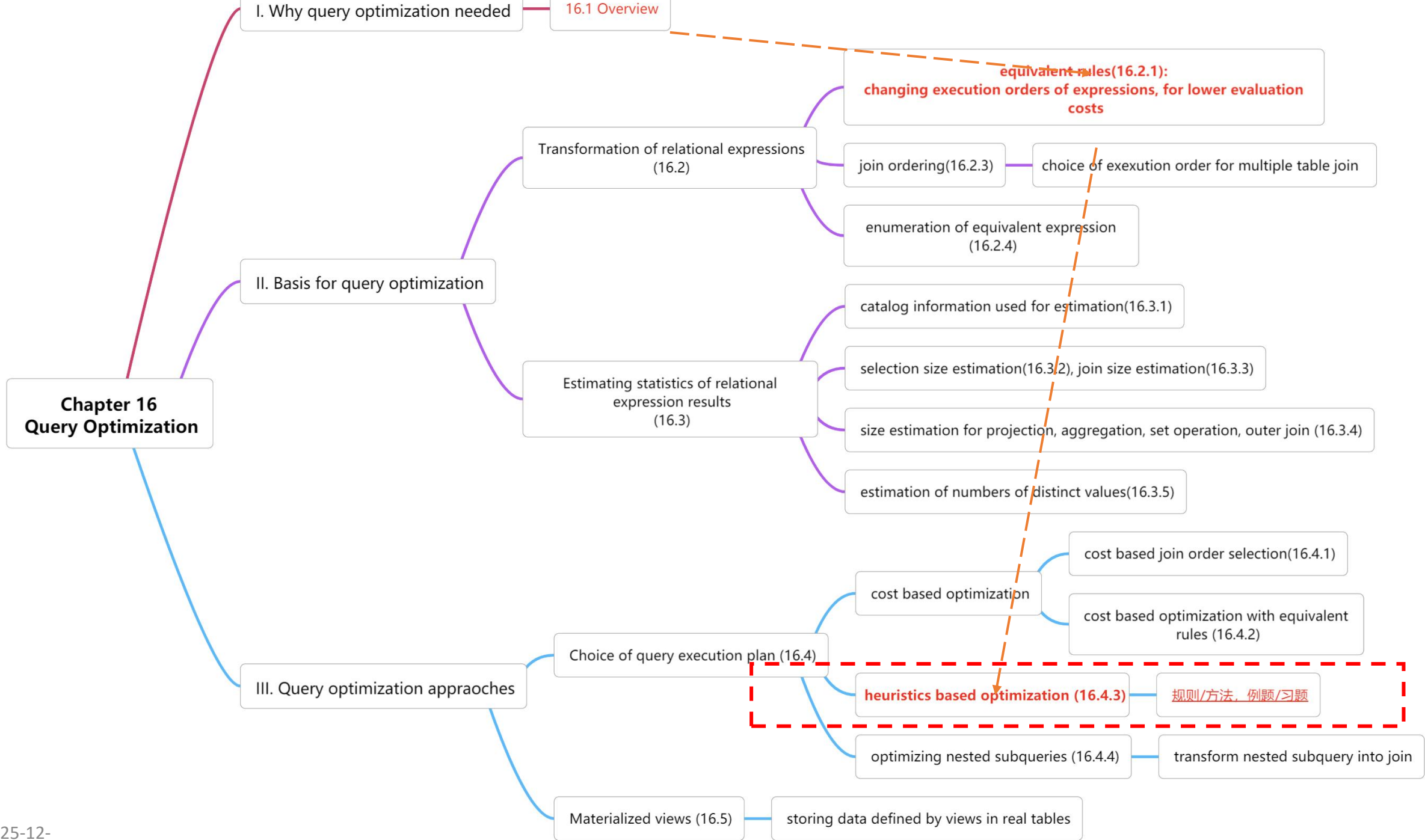
- **Materialization**: generate results of an expression whose inputs are relations or are already computed, **materialize (store) it on disk**. Repeat.
- **Pipelining**: pass on tuples to parent operations even as an operation is being executed

Chapter 16 Query Optimization

- 关系代数等价变换规则
- 启发式查询优化方法

- 例题/作业

- ▣ 查询需求 → SQL语句 → 优化后的查询树（启发式查询优化方法）
- ▣ 注意
 - 多表查询SQL语句中，from部分使用连接操作，代替where中的连接条件
 - 利用选择下移、投影下移，简化查询
 - 避免查询树中出现笛卡尔积操作
 - 参与连接操作的左右子树在连接操作之前，先使用投影操作筛选出后面要用到属性



Part 5 TRANSACTION MANAGEMENT

- Part 5
 - ▣ Chapter 17 Transactions
 - ▣ Chapter 18 Concurrent Control
 - ▣ Chapter 19 Recovery System
- DBS中，从DBMS的角度，用户对DB的访问体现为DBS中1个或多个数据库**事务(transaction)**的执行
 - ▣ select, insert, delete, update
 - ▣ transaction类似于process, thread
- 事务是DBS中应用程序/数据库应用系统的基本逻辑单位，也是DBMS管理DBS运行的基本单位, §17.1/17.2
- 多用户DBS中，多个用户对DB的并发访问体现为DBS中多个数据库事务的并发执行
- 事务是动态的，具有一定生命周期
 - ▣ 事务具有多种状态，事务的执行体现为各状态间的转换过程, §17.4, Fig.17.1

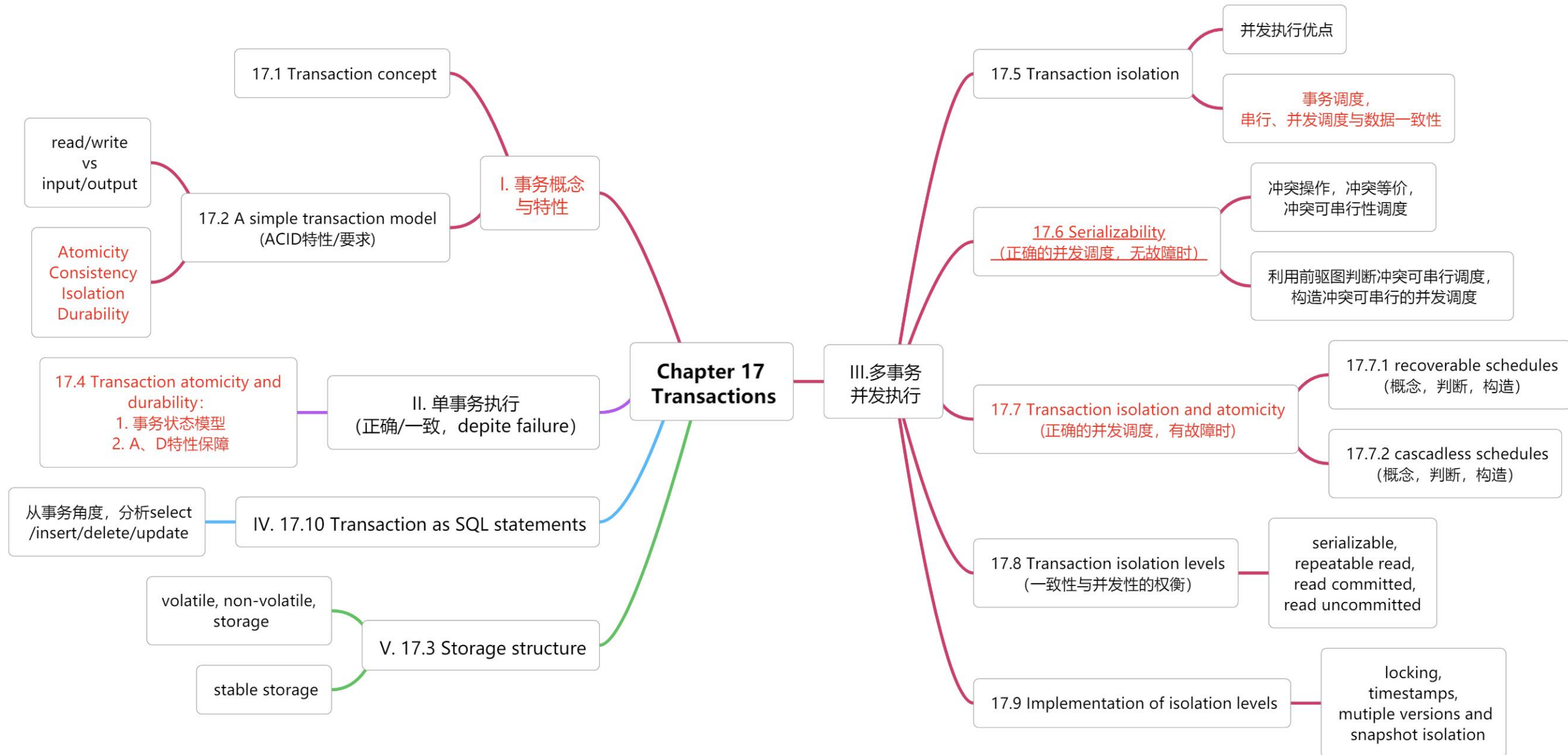
TRANSACTION MANAGEMENT

- 在DBS中，事务执行时，特别是当多个事务对共享数据进行并发访问时，为维护DBS系统中数据的正确性(一致性integrity), 事务必须满足一定的约束条件，表现为事务的4个基本特征(ACID), §17.1/17.2/17.4/17/5
 - 原子性(atomicity), 一致性(consistency), 独立性/隔离性(isolation), 永久性/持续性/操作结果永久保持性 (durability)
- DBMS中的log-based recovery-management component负责保障事务的原子性atomicity (§19)
- 事务设计者/DBS应用程序编程者负责保障事务的一致性
- DBMS中的concurrency-control component负责保障事务的独立性 (§18)
 - isolation level
 - locking, timestamp, multiple-granularity, multiple-version
- DBMS中的recovery-management component负责保障事务的永久性 (§19)

TRANSACTION MANAGEMENT

- 当DBS中存在多个并发事务时，DBMS通过事务调度对各事务进行并发控制 (§18.4)，以保证并发事务的运行结果正确性
 - ▣ DBMS中的concurrency-control component依据事务调度可串行性 (Serializability) 的基本原理，对事务进行并发控制和调度,§18.6
 - ▣ 具体实现技术
 - 1. 基于锁的并发控制技术 (§18.1) —— Reader/Writer Model
 - 2. 多粒度控制技术 (§18.3)
 - ~~多版本控制 (§18.6)~~
 - ~~【略：3. 隔离级别 —— tradeoff between —— 一致性/正确性与访问效率~~
 - ~~4. 基于时间戳的并发控制技术 (§18.4)~~
 - ~~5. 基于验证的并发控制技术 (§18.5)】~~
- ~~此外，DBMS的concurrency-control component还需要对多个并发执行的事务进行死锁处理 (§18.2)~~

Chapter 17 Transactions



Transactions

- 事务定义, 事务组成, 事务与应用程序的关系
- 事务ACID特性, ACID的定义, ACID保障机制
 - DBMS中的recovery-management component负责保障事务的原子性(§19)
 - 事务设计者/DBS应用程序编程者 + integrity机制负责保障事务的一致性
 - DBMS中的concurrency-control component负责保障事务的独立性(§18)
 - isolation level//隔离级别
 - locking, multiple-granularity, multiple-version, timestamp, ...
 - DBMS中的recovery-management component负责保障事务的永久性(§19)

Transaction

e.g. *name salary* in *Instructor*

data accesses on data item **x** and **y** issued by T_i ,
e.g. *select, insert, delete, update, ...*

DBMS

x_i, y_i

local buffer for transaction T_i

逻辑读写

read(x)

write(y)

B_x

B_y

...

B_z

disk buffer

物理读写

input(X)

output(Y) / *reflect* / 持久化 when checkpoint or commit

DB file on disk

B_x

B_y

...

B_z

B_w

B_u

B_v

...

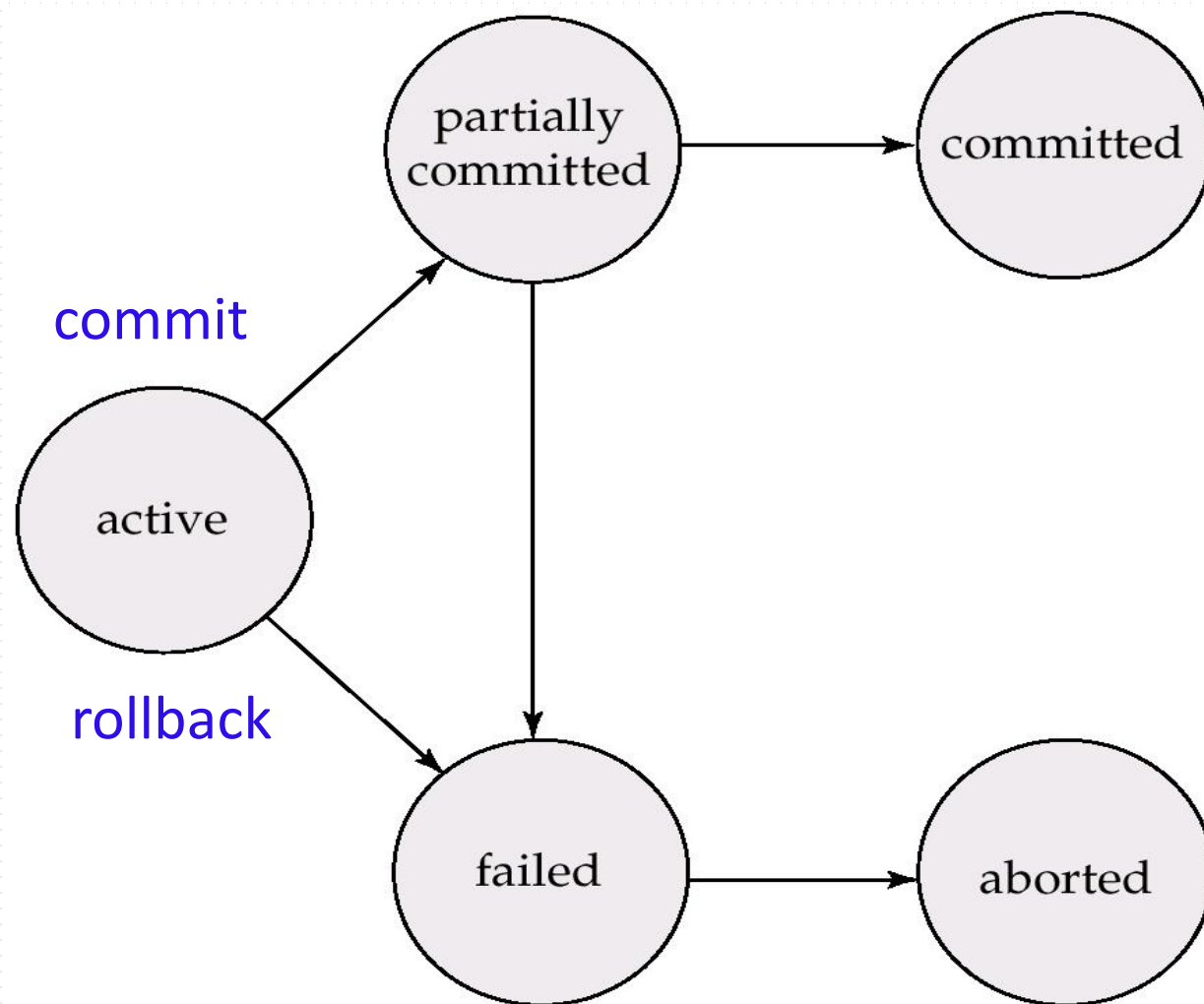
B_q

B_r

Fig.17.0.3 *read* and *write* operations

Transactions

- 事务状态及转换【大致了解概念】
 - ▣ 事务执行所经历各阶段，各状态下事务所做工作
 - ▣ commit, rollback, abort



Transaction

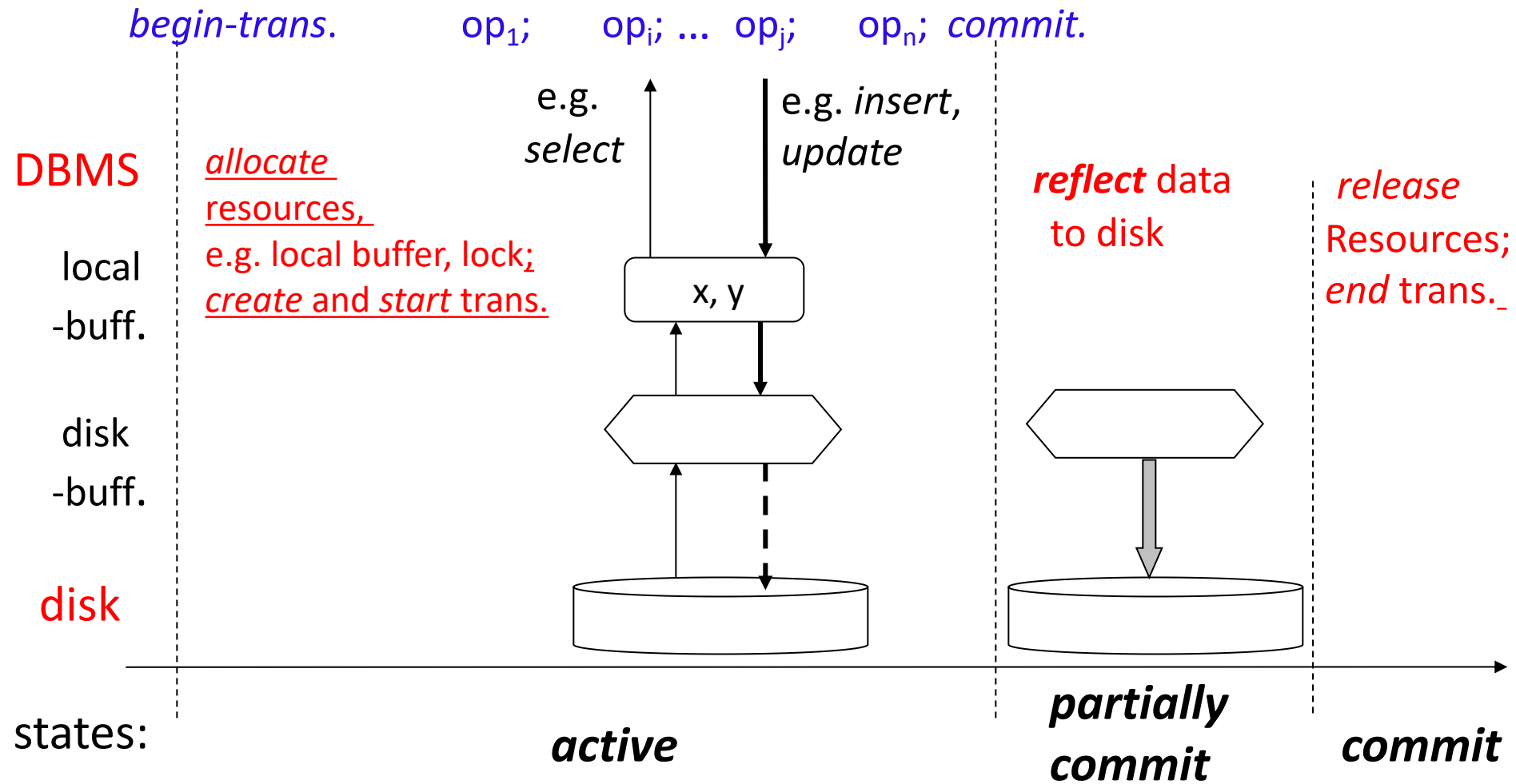


Fig.17.0.5 A successful/*committed* transaction

Transaction

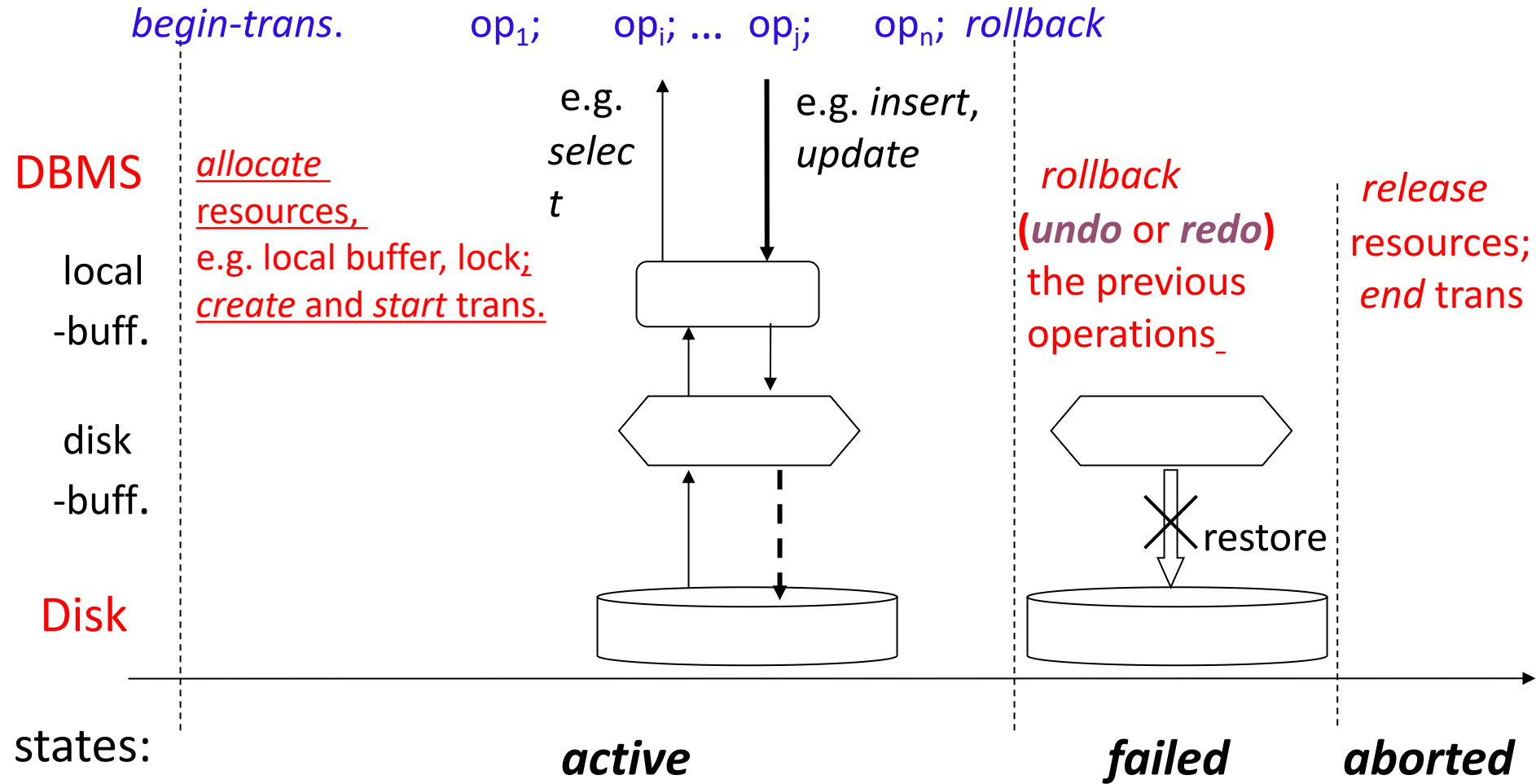


Fig.17.0.6 A *rollback* transaction

Concurrent Scheduling

- (17.5 Transaction Isolation)
- 事务的串行调度、并发调度
- 数据库系统中，多个事务的**串行执行**可以保证事务的ACID特性，但执行效率低
- 事务并发执行，事务操作的调度顺序不当，可能导致的数据库不一致
 - 原因：没有做到冲突操作串行执行
- 并发事务中的冲突操作
- 调度S与调度S'冲突等价 (conflict equivalent)
- Conflict Serializability
 - a (possibly concurrent) schedule S is **serializable** if it is equivalent to a serial schedule
 - 将对调度S正确性的判断转换为对S可串行性的判断

Concurrent Scheduling

- 事务并发控制原理：事务调度可串行化
 - 构造可串行化(Serializability)的并发事务调度，使得相互冲突的操作串行执行，非冲突操作并发/交织执行，“执行效果”等价于串行调度（从而在无故障发生时，调度的执行结果是正确的，可保证ACID特性），但执行效率更高
- 并发调度的Serializability
 - 判断并发调度是否正确标准
- 并发调度S的可串行性判断测试 (Testing for serializability)
 - 方法1：根据定义
 - 方法2：前驱图
 - 根据图中是否有环路，判断S是否冲突可串行
 - 从无环路前驱图出发，构造与S等价的串行调度

【发生故障时，并发调度】

- Two types of schedules
 - ▣ recoverable scheduling
 - ▣ cascadeless scheduling
- Every cascadeless schedule is also recoverable, but a recoverable may not be a cascadeless schedule
- 【略】 ~~17.8 Transaction Isolation Level/Weak Levels of Consistency~~
 - ▣ ~~Serializable~~
 - ▣ ~~Repeatable read~~
 - ▣ ~~Read committed~~
 - ▣ ~~Read uncommitted~~

为提高并发事务执行效率，放松一致性要求：

- 1) 允许一定程度的“读-写”、“写-读”冲突
- 2) 仍然严格保证无冲突的“写-写”

例题/作业

- 构造并发调度
- 根据定义或前驱图，判断并发调度是否可串行
- 判断并发调度是否是可恢复recoverable、无级联回滚cascadeless

Chapter 18 Concurrency Control

- 根据并发调度可串行化原理, 采用以下机制控制多个事务对共享数据的并发访问: (Reader-writer model)
 - locking, multi- granularity schemes, multi-version schemes
- Lock-based Protocol
 - 锁类型S-lock(Q), X-lock(Q), lock_S/lock_X, unlock_S/unlock_X
 - 2PL结构
 - growing阶段, shrinking阶段, 锁点lockpoint
 - enhanced 2PL: strict 2PL, rigorous 2PL
- 2PL特点
 - 2PL可以构造conflict serializable并发调度, 但无法避免级联回滚, 无法避免死锁
 - 严格/强制2PL可以构造~~cascadeless~~, ~~recoverable~~ and ~~serializable~~ 的并发调度; 有可能导致死锁、饿死
 - 2PL构造的并发调度等价于按照锁点排列的串行调度

Concurrency Control

- 例题/作业
 - 判断调度是否满足2PL
 - 给定多个事务，构造符合2PL的调度
- 死锁检测与恢复【略，本次不考】
 - deadlock handling原理：死锁发生的四个必要条件
 - 检测
 - 等待图wait-graph，有环路→有死锁
 - 恢复
 - 回滚、撤销处于环路中的事务

Concurrency Control

■ 多粒度锁协议

□ 不同粒度的数据对象

- 逻辑对象：属性值、属性值集合、元组、关系、索引项、整个索引、数据库
- 物理对象：数据页page、索引页、数据块extent, DB file

□ 施加

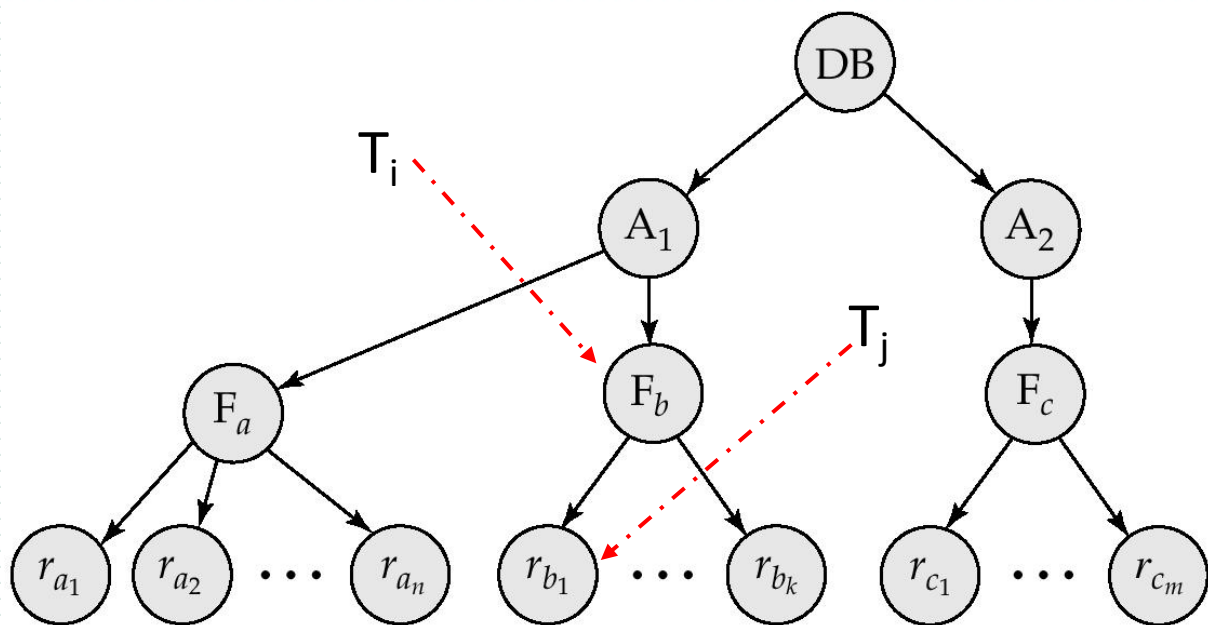
- S、X
- 意向锁IS、IX、SIX

□ 锁相容性矩阵：加锁规则

■ 自上而下加锁，自下而上解锁

■ 例题/作业

- ### □ 为并发事务添加DB、table、tuple级的S/X锁和意向锁



Compatibility Matrix with Intention Lock Modes

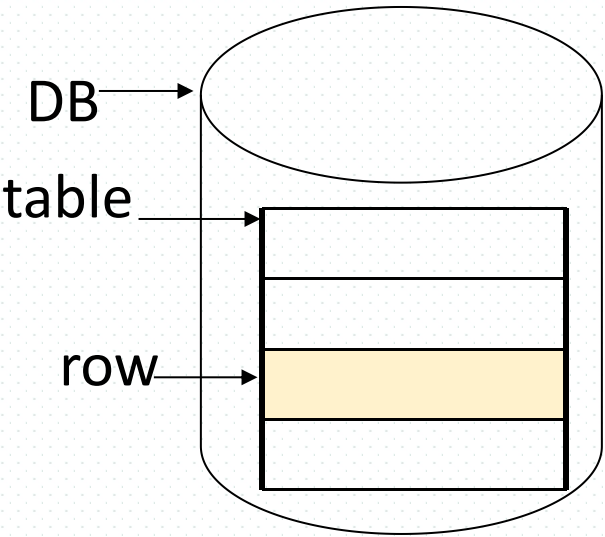
现有T_j
请求T_i

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

X:
update
insert
bulk insert

S, X: 局部, 元组/行级锁
IS, IX, SIX: 整体

E.g.
T_i want to access on the
table **student**;
T_j is now accessing on
student



IS on DB
IS on table
S on tuple

```
select ID  
from student  
where age>0
```

IS/IX on DB
S/X on table

```
select ID  
from student  
  
alter table  
create index
```

IX/SIX on DB
IX on table
X on tuple

```
update student  
set ID:=ID+1  
where age >20  
  
delete  
from student  
where age>0
```

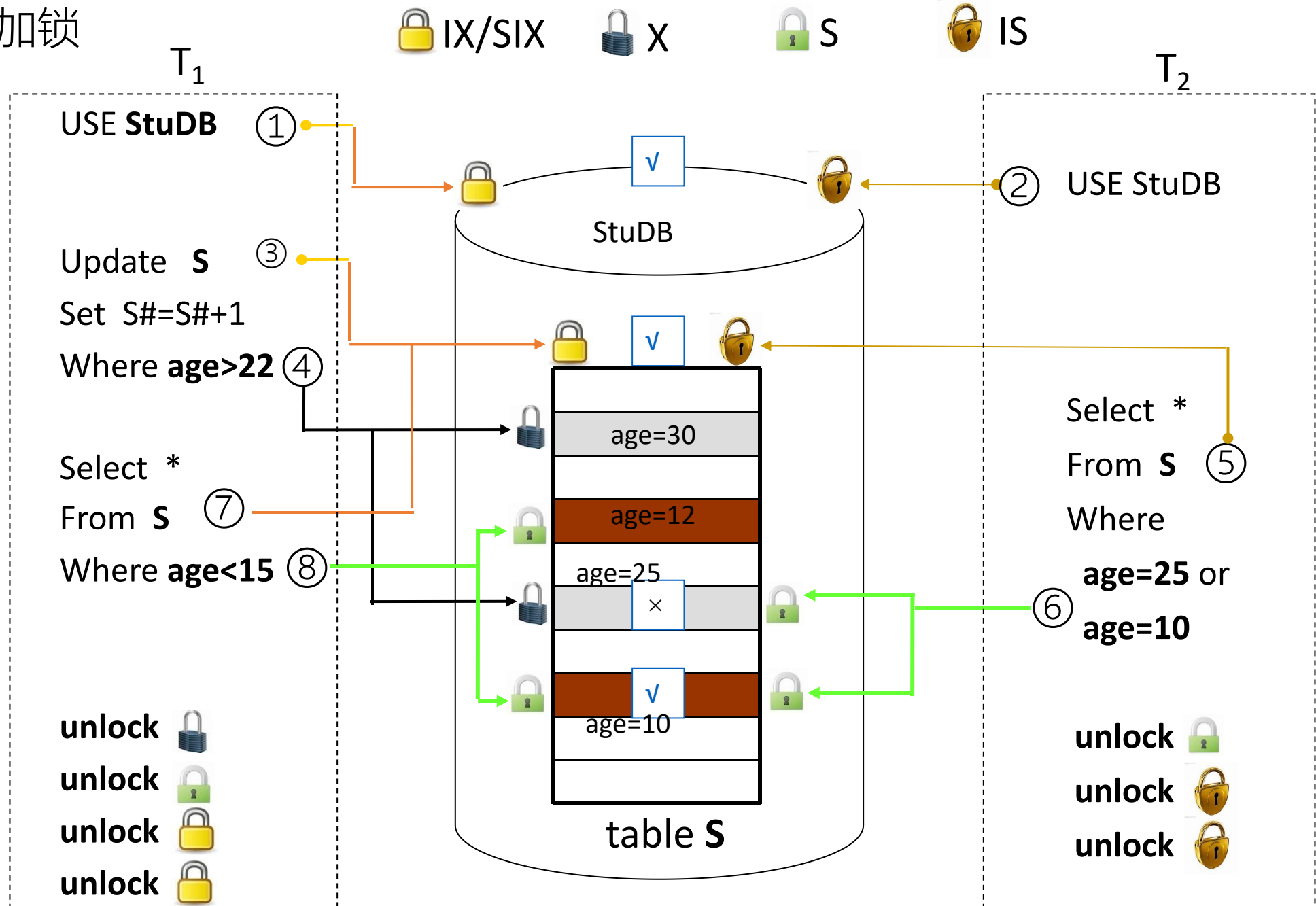
IX/SIX on DB
X on table

```
update student  
set ID:=ID+1  
  
delete  
from student
```

IX/SIX on DB
X on table

```
insert into student  
values(ID, name,...)
```

自上而下加锁



Chapter 19 Recovery

- Three types of failures may occur in DBS
 - ▣ 1. Transaction failure
 - logical errors, system errors(e.g. deadlock)
 - ▣ 2. System crash
 - ▣ hardware malfunction, bugs in DBS software or operating systems
 - e.g. power failure or other hardware failures
 - ▣ 3. Disk (storage medium failure)——介质故障
- Failure handling/故障处理机制
 - ▣ 1. transaction failure, system crash: **log-based recovery** scheme
 - ▣ 2. disk failure: 备份/backup (e.g. RAID) , 灾备等机制

19.3 Recovery and Atomicity

crash→数据不一致, 违反原子性→ recovery的必要性

■ Log Records

- $\langle T_i \text{ start} \rangle$, $\langle T_i, X, V1, V2 \rangle$, $\langle T_i \text{ commit} \rangle$
- 事务主动回滚rollback带来的补偿日志: $\langle T_i, X, V1 \rangle$ $\langle T_i \text{ abort} \rangle$
- 检查点日志 $\langle \text{checkpoint}(k), L \rangle$

■ Two types of database modification

- deferred modification (延时更新), immediate modification (即时更新)

■ Recovery by undo/撤销 and redo/重做 operations

- **undo** using a log record $\langle T, X, V1, V2 \rangle$ sets the data item X specified in the record to the old value V1, 保证事务原子性
- **redo** using a log record $\langle T, X, V1, V2 \rangle$ sets the data item X specified in the record to the new value V2, 保证事务持久性

Recovery and Atomicity

■ Recovery operations due to system crash

- ❑ 【commit之后crash】 if there is a system crash after T commits, T's updates can be **redone (redo操作)**
 - based on $\langle T, X, V1, V2 \rangle$, rewrite new value X2 into database file on disk
- ❑ 【commit之前crash】 if a system crash occurs before the log record $\langle T, committed \rangle$ is output to the stable storage, i.e. before T has committed, T will be **rolled back (undo操作)**
 - based on $\langle T, X, V1, V2 \rangle$, undo the modification to X, and recovery X to V1

■ 多个串行事务，根据日志，判断采取的恢复动作、恢复结果

$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$	$\langle T_0 \text{ start} \rangle$
$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$	$\langle T_0, A, 1000, 950 \rangle$
$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$	$\langle T_0, B, 2000, 2050 \rangle$
	$\langle T_0 \text{ commit} \rangle$	$\langle T_0 \text{ commit} \rangle$
	$\langle T_1 \text{ start} \rangle$	$\langle T_1 \text{ start} \rangle$
	$\langle T_1, C, 700, 600 \rangle$	$\langle T_1, C, 700, 600 \rangle$
		$\langle T_1 \text{ commit} \rangle$
(a)	(b)	(c)

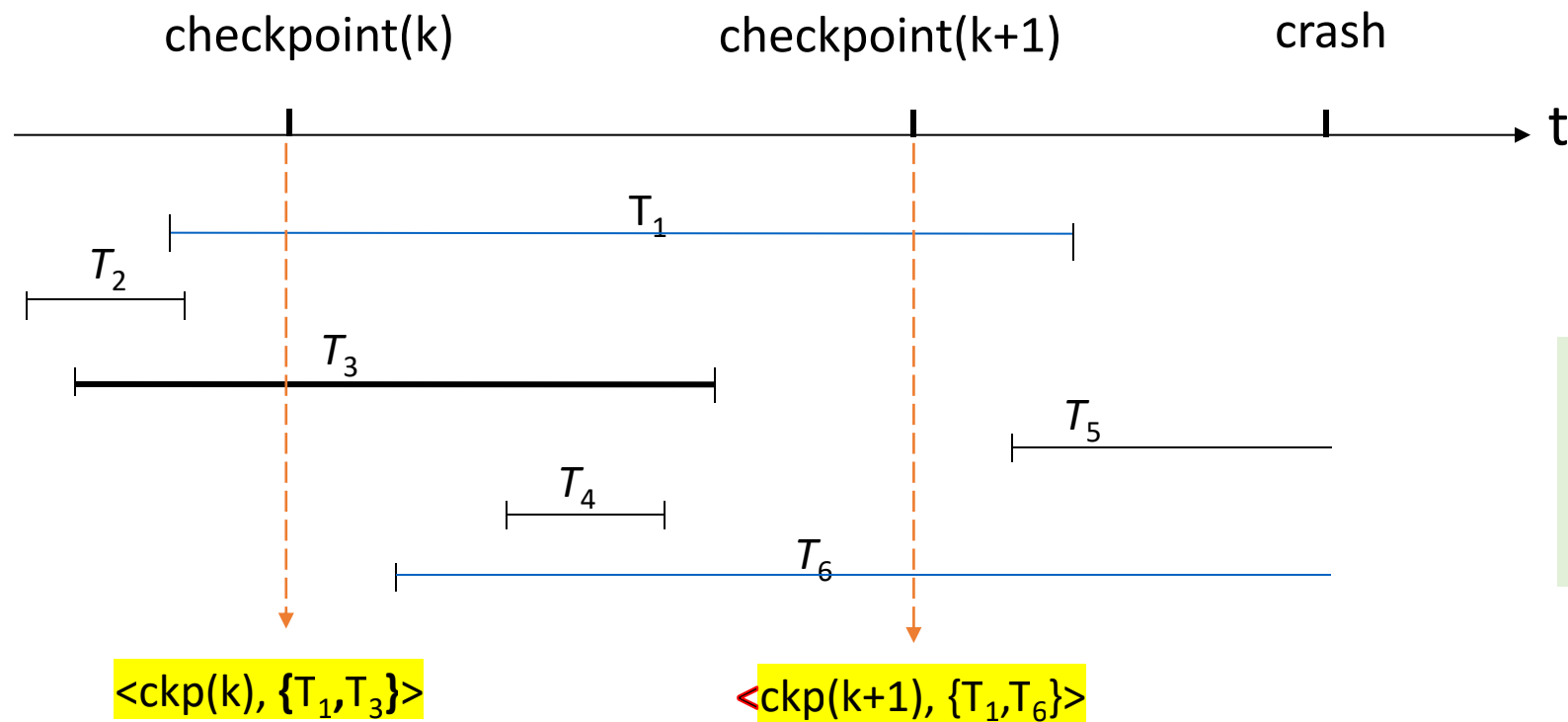
Checkpoints

■ Checkpoint actions

- ❑ 1. 【日志计入stable storage】 output all **log records** currently residing in main memory onto the log file on stable storage
- ❑ 2. 【**modified数据批量写回DB文件**】 **output/reflect** all modified buffer blocks in the disk buffer prior to the checkpoints to the disk!
 - 定期将内存中的log和disk buffer中修改后的数据块写回外设磁盘DB文件中
 - 优点：1. 随机写→批量写，降低I/O负荷；2. 节省buffer
- ❑ 3. 【生成检测点日志】 output onto stable storage a log record of the form **<checkpoint L>**, where L is a list of concurrent transactions active at the time of the checkpoint

■ 利用检查点，判断并发事务采取的恢复动作

- ❑ 情况1：最近检查点之前完成的事务(commit/rollback日志在该检查点之前)，无需恢复动作
- ❑ 情况2：最近检查点时仍活跃，或最近检查点之后开始的事务，需要redo/undo
 - for T_k suc that have no $\langle T_k \text{ commit} \rangle$ record or $\langle T_k \text{ abort} \rangle$ record in the log, execute $\text{undo}(T_k)$
 - for T_k such that either the record $\langle T_k \text{ commit} \rangle$ or the record $\langle T_k \text{ abort} \rangle$ appears in the log, execute $\text{redo}(T_k)$



undo: T_5, T_6
redo: T_1
ignored: T_2, T_3, T_4

log:

[头]: $\langle T_2 \text{ start} \rangle; \dots; \langle T_3 \text{ start} \rangle; \dots; \langle T_1 \text{ start} \rangle; \dots; \langle T_2 \text{ commit} \rangle; \dots; \langle \text{ckp}(k), \{T_1, T_3\} \rangle; \dots;$
 $\langle T_6 \text{ start} \rangle; \dots; \langle T_4 \text{ start} \rangle; \dots; \langle T_4 \text{ commit} \rangle; \dots; \langle T_3 \text{ commit} \rangle; \dots; \langle \text{ckp}(k+1), \{T_1, T_6\} \rangle; \dots;$
 $\langle T_5 \text{ start} \rangle; \dots; \langle T_1 \text{ commit} \rangle; [\text{尾}]$



Fig. 19.0.18 Checkpoints and logs for concurrent transactions

19.4 Recovery Algorithms (for Concurrent Transactions)

- 两种情况下的recovery算法
 - ▣ 1. use log records for recovery from transaction failures, issued by rollback
 - 事务主动回滚, log文件中有补偿/回滚日志 $\langle T_i, X, V1 \rangle \langle T_i \text{ abort} \rangle$
 - ▣ 2. use a combination of most recent checkpoint and log records to recovery from a system crash
 - 发生外部故障

Recovery Actions due to Rollback

- 【事务主动rollback时，采取的动作】 Rollback of a transaction T_i is performed as follows
 - ▣ 1. scan log backwards from the end, and for each log record of T_i of the form $\langle T_i, X_j, V_1, V_2 \rangle$
 - (1) perform the undo by writing V_1 to X_j , //undo操作，恢复为旧值 V_1
 - (2) write a log record $\langle T_i, X_j, V_1 \rangle$, such log records are called **compensation log records**
//为被回滚事务，生成补偿日志
 - ▣ 2. once the record $\langle T_i \text{ start} \rangle$ is found, stop the scan and write the log record $\langle T_i \text{ abort} \rangle$
//标记被回滚事务为“夭折abort”

Recovery After a System Crash

- Recovery actions take place in two phases: redo phase, undo phase
 - ▣ step1. redo phase——对已提交事务redo, 并构造undo-list
 - ▣ step2. undo phase——根据undo-list, 回滚未提交事务

Example 1: 根据并发事务日志, 判断故障恢复动作和恢复后的数据项值

T_0 由于 transaction failure 导致主动 rollback, 完成 **rollback** 后遇到系统 crash; 系统重启后, 完成 recovery from crash, 并发执行过程: I) 正常执行 II) rollback III) **crash**, and recovery from crash

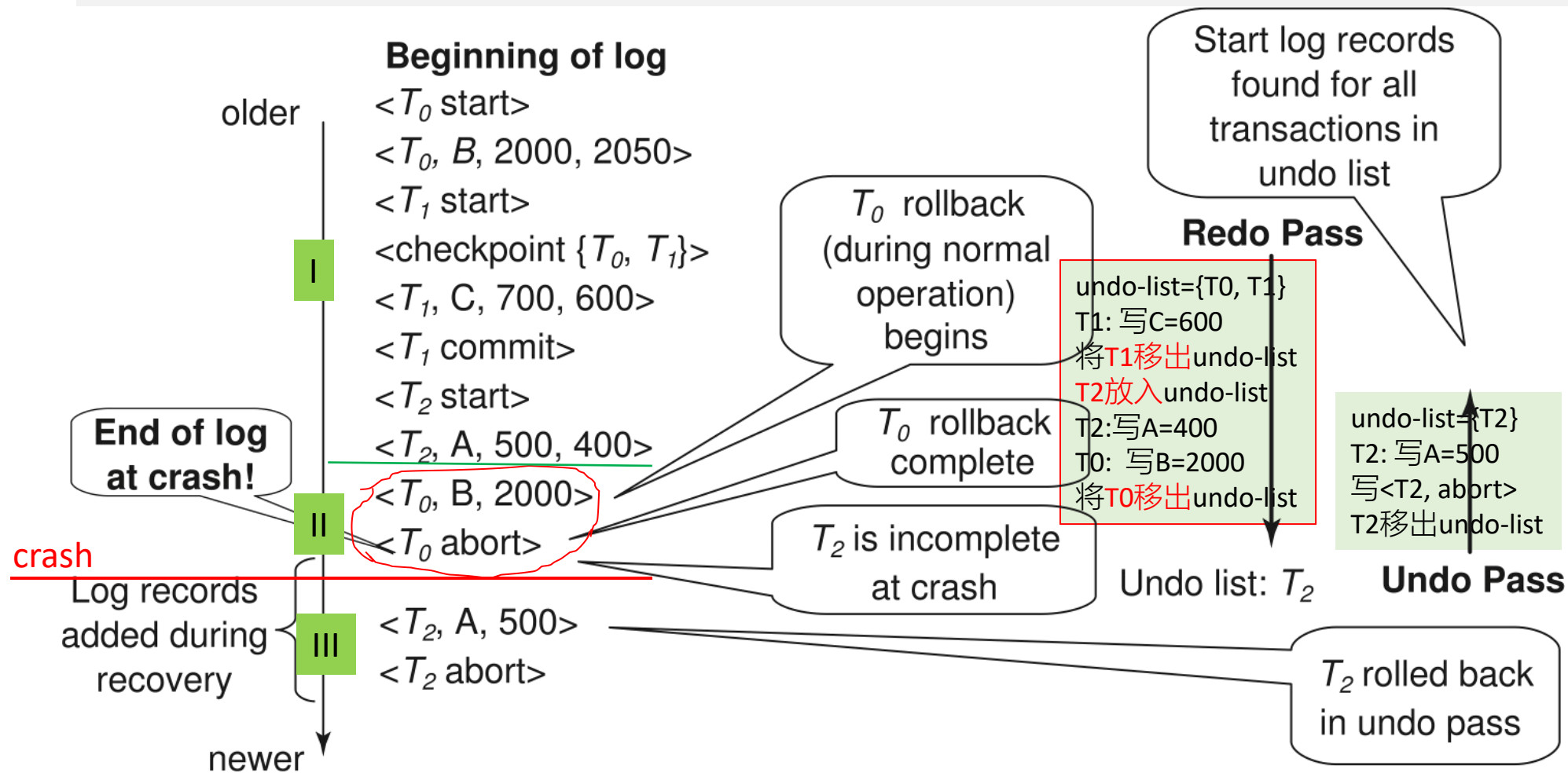


Fig. 19.5 Example of Recovery: Log records

Example 1

- 根据并发事务日志，判断故障恢复动作和恢复后的数据项值
 - 两类恢复：1. 事务主动回滚rollback; 2. crash导致recovery
- 解题方法
 - 根据日志Fig. 19.5，画出事务并发执行流程Fig. 19.5-1
 - 根据事务并发执行流程，判断对并发事务应采取的动作、恢复后的数据项值
- Recovery动作
 - 最近checkpoint时仍活跃，crash之前已经正常提交commit的事务 T_1 ，需要redo，以保证持久性
 - 最近checkpoint时仍活跃，crash之前已经rollback主动回滚，完成recovery的事务 T_0 ，需要redo，以保证持久性
 - 最近checkpoint时仍活跃，crash之后没有commit或rollback的事务 T_2 ，需要undo，以保证原子性
 - 最近checkpoint之前已经commit/rollback，即checkpoint时不活跃的事务 T_4 （注：补充添加），不考虑(ignored)

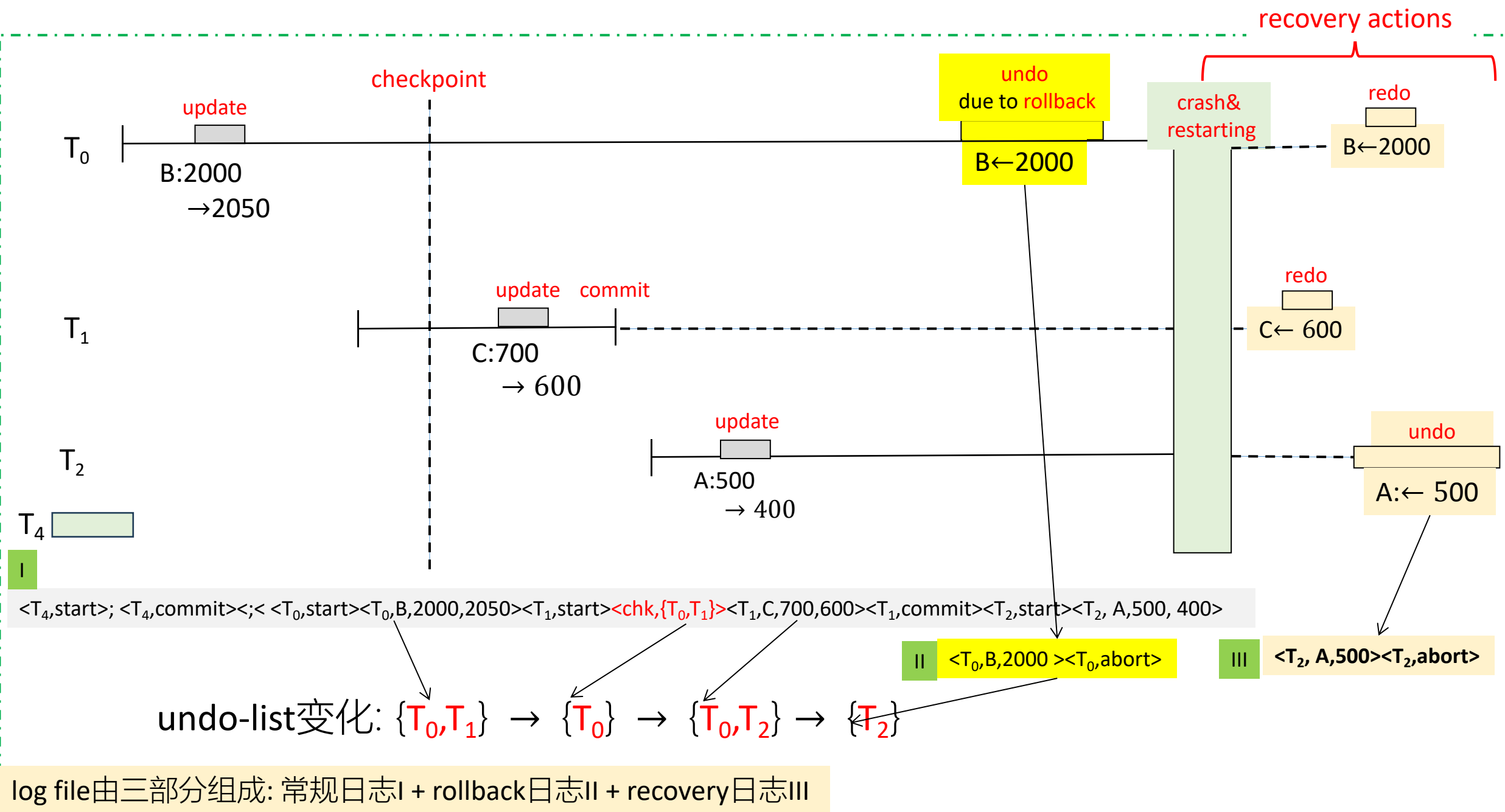


Fig. 19.5-2 Execution of Concurrent Transactions

Example 2

- 参见: 
- Fig. 19.0.18 shows the time scale for six concurrent T_1 , T_2 , T_3 , T_4 , T_5 and T_6 , two checkpoints are set
- After a system crash occurs, what recovery operations (i.e. redo, undo, or ignored) should the DBMS recovery subsystem apply to each transaction?
 - ▣ assuming that immediate database modification technique is used for log-based recovery

Example 3: 根据并发调度构造日志

- The initial values of data items A and B are 1000 and 2000 respectively
- If immediate-modification is employed, give the log that is constructed by the recovery scheme and describes the concurrent executing of transactions T_1 and T_2 , as shown in Fig. 19.0.21
-

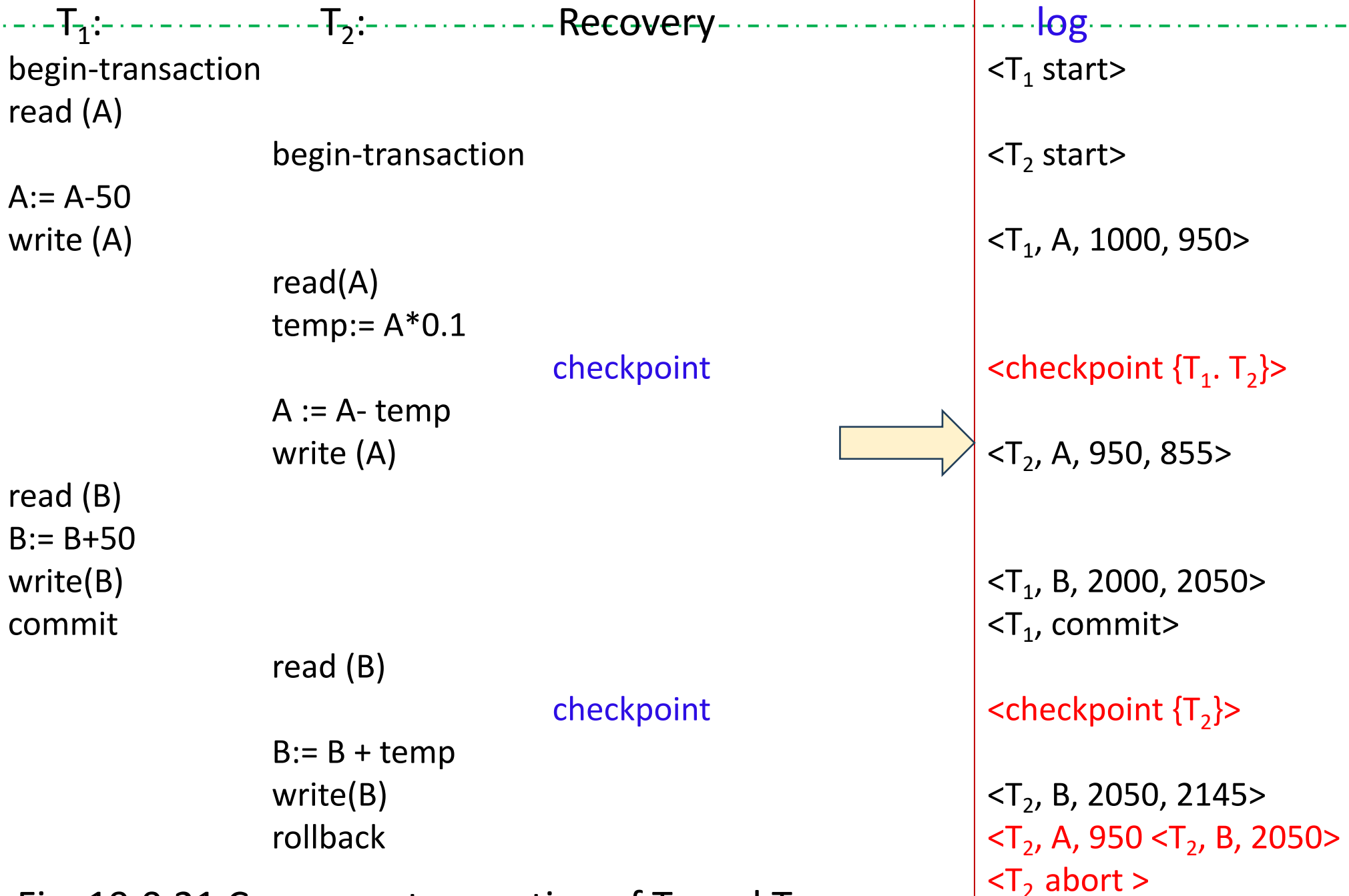


Fig. 19.0.21 Concurrent executing of T_1 and T_2

■ Example 3 + Example 2

- 根据并发调度，给出日志记录(e.g. Fig. 19.0.21)
 - concurrent schedule → log file
- 根据并发调度，构造并发执行轨迹 (e.g. Fig. 19.5-2)，判断需要恢复的事务，以及采取的恢复动作redo/undo
 - log file → transactions to be recovered, recovery actions
- 根据恢复动作，判断恢复后的数据项值
 - recovery actions → the recovered values of data items

■ 恢复动作

- 最近checkpoint时仍活跃，crash之前已经正常提交/commit的事务，需要redo
- 最近checkpoint时仍活跃，crash之前已经rollback主动回滚，完成recovery的事务，需要redo
- 最近checkpoint时仍活跃，crash之后没有commit或rollback的事务，需要undo
- 最近checkpoint之前已经commit/rollback，即最近checkpoint时不活跃的事务，不考虑(ignored)

17-19章 例题

- 数据库系统原理-17-19-DBS事务管理例题-v1-2025



数据库系统原理-17-19-事务管理例题-v1-2025.tar



Thanks for your
attention



北京邮电大学